Muffix Sassif – TRD

Andrianov, Lepeshov, Shulyatev

November 3, 2026



Содержание

1 Геометрия **2**
- 1.1 3D . . . 2
- 1.2 База 1 - вектор . . . 2
- 1.3 База 2 - прямая . . . 2
- 1.4 База 3 - окружность . . . 3
- 1.5 Выпуклая оболочка . . . 3
- 1.6 Задача 16 . . . 3
- 1.7 Калиперы . . . 4
- 1.8 Касательные к двум многоугольникам . . . 4
- 1.9 Касательные к многоугольнику . . . 4
- 1.10 Лежит ли точка в многоугольнике . . . 4
- 1.11 Минимальная покрывающая окружность . . . 4
- 1.12 Пересечение полуплоскостей . . . 4
- 1.13 Проверка на пересечение отрезков . . . 5
- 1.14 Сумма Минковского . . . 5

2 Графы **5**
- 2.1 2-SAT . . . 5
- 2.2 L-R Flow . . . 5
- 2.3 WeightedMatching . . . 6
- 2.4 Венгерский алгоритм . . . 7
- 2.5 Вершинная двусвязность . . . 7
- 2.6 Диниц . . . 7
- 2.7 КСС . . . 8
- 2.8 Кактус . . . 8
- 2.9 Минкост . . . 8
- 2.10 Мосты . . . 9
- 2.11 Паросочетания . . . 9
- 2.12 Пересечение матроидов . . . 9
- 2.13 Простой путь с заходом . . . 10
- 2.14 Точки сочленения . . . 10
- 2.15 Эйлеров цикл . . . 10

3 ДП **10**
- 3.1 CHT . . . 10
- 3.2 Li Chao . . . 11
- 3.3 SOS-dp . . . 11
- 3.4 UnlimitedKnapsack . . . 11
- 3.5 НВП . . . 11

4 Деревья **11**
- 4.1 Centroid . . . 11
- 4.2 HLD . . . 12
- 4.3 Link-cut . . . 12

5 Другое **13**
- 5.1 Fast hashmap . . . 13
- 5.2 Fast mod . . . 13
- 5.3 Fast sort . . . 13
- 5.4 attribute_packed . . . 13
- 5.5 custom_bitset . . . 13
- 5.6 Аллокатор Копелиовича . . . 14
- 5.7 Альфа-бета отсечение . . . 14
- 5.8 Отжиг . . . 14

6 Математика **14**
- 6.1 AdivB cmp CdivD . . . 14
- 6.2 Berlecamp . . . 14
- 6.3 FFT . . . 14
- 6.4 Floor Sum, Min Rem . . . 15
- 6.5 GCD LCM свёртки . . . 16
- 6.6 Interpolation . . . 16
- 6.7 Min25 . . . 16
- 6.8 NTT . . . 17
- 6.9 OR XOR AND свёртки . . . 18
- 6.10 convMod . . . 18
- 6.11 sqrt mod . . . 19
- 6.12 Гаусс . . . 19
- 6.13 Диофантовы уравнения . . . 19
- 6.14 КТО . . . 19
- 6.15 Код Грея . . . 19
- 6.16 Линейное решето . . . 20
- 6.17 Миллер Рабин . . . 20
- 6.18 Подсчёт прогулок . . . 20
- 6.19 Ро-Поллард . . . 20
- 6.20 Чудо Формулы . . . 20

7 Строки **21**
- 7.1 Z, префикс, Манакер, min rotation . . . 21
- 7.2 eertree . . . 21
- 7.3 Ахо-Корасик . . . 22
- 7.4 Муффиксный Сассив . . . 22
- 7.5 Суффиксный автомат . . . 22

8 Структуры данных **22**
- 8.1 Disjoint Sparse Table . . . 22
- 8.2 Segment Tree Beats . . . 23
- 8.3 ДД по неявному . . . 23
- 8.4 ДД . . . 23
- 8.5 Персистентное ДД по неявному . . . 24
- 8.6 Персистентное ДО . . . 24
- 8.7 Спарсы . . . 24
- 8.8 Фенвик . . . 24

1 Геометрия

1.1 3D

```
struct Pt {
  dbl x, y, z;
  Pt() : x(0), y(0), z(0) {}
  Pt(dbl x, dbl y, dbl z) : x(x), y(y), z(z) {}
  Pt operator-(const Pt& o) const {
    return {x - o.x, y - o.y, z - o.z};
  }
  Pt operator+(const Pt& o) const {
    return {x + o.x, y + o.y, z + o.z};
  }
  Pt operator/(const dbl& a) const {
    return {x / a, y / a, z / a};
  }
  Pt operator*(const dbl& a) const {
    return {x * a, y * a, z * a};
  }
  Pt cross(const Pt& o) const {
    dbl nx = y * o.z - z * o.y;
    dbl ny = z * o.x - x * o.z;
    dbl nz = x * o.y - y * o.x;
    return {nx, ny, nz};
  }
  dbl dot(const Pt &o) const {
    return x * o.x + y * o.y + z * o.z;
  }
  bool operator==(const Pt& o) const {
    return abs(x - o.x) < EPS && abs(y - o.y) < EPS && abs(z -
      o.z) < EPS;
  }
  dbl dist() {
    return sqrtl(x * x + y * y + z * z);
  }
};

struct Plane {
  dbl a, b, c, d;
  Plane(dbl a_, dbl b_, dbl c_, dbl d_) : a(a_), b(b_), c(c_),
      d(d_) {
    dbl z = sqrtl(a * a + b * b + c * c);
    if (z < EPS) return;
    a /= z, b /= z, c /= z, d /= z;
  }
  dbl get_val(const Pt &p) const {
    // НЕ СТАВИТЬ МОДУЛЬ
    return a * p.x + b * p.y + c * p.z + d;
  }
  dbl dist(const Pt &p) const {
    return abs(get_val(p));
  }
  bool on_plane(const Pt &p) const {
    return dist(p) / sqrtl(a * a + b * b + c * c) < EPS;
  }
  Pt proj(const Pt &p) const {
    dbl t = get_val(p) / (a * a + b * b + c * c);
    return p - Pt(a, b, c) * t;
  }
};

bool on_line(Pt p1, Pt p2, Pt p3) {
  return (p2 - p1).cross(p3 - p1) == Pt(0, 0, 0);
}

Plane get_plane(Pt p1, Pt p2, Pt p3) {
  Pt norm = (p2 - p1).cross(p3 - p1);
  Plane pl(norm.x, norm.y, norm.z, 0);
  pl.d = -pl.get_val(p1);
  return pl;
}

// две точки прямой пересечения на расстоянии 1
pair<Pt, Pt> intersect(Plane pl1, Plane pl2) {
  Pt n1(pl1.a, pl1.b, pl1.c), n2(pl2.a, pl2.b, pl2.c);
  Pt dir = n1.cross(n2);
  dbl len2 = dir.dot(dir);
  if (len2 < EPS * EPS) return {Pt(), Pt()}; // параллельны
  Pt p = (n1 * pl2.d - n2 * pl1.d).cross(dir) / len2;
  return {p, p + dir / sqrtl(len2)};
}

// угол между векторами в [0, pi]: сонаправленные -> 0, против
    оположные -> pi
// угол между прямыми с направляющими p1, p2: min(a, pi - a)
dbl get_ang(Pt p1, Pt p2) {
  return atan2l(p1.cross(p2).dist(), p1.dot(p2));
}

// любой перпендикулярный вектор
Pt vector_perp(Pt v) {
  if (abs(v.x) > EPS || abs(v.y) > EPS)
    return {v.y, -v.x, 0};
  return {v.z, 0, -v.x};
}

// плоскость через точку p перпендикулярная вектору v
Plane plane_perp(Pt p, Pt v) {
  return Plane(v.x, v.y, v.z, -v.dot(p));
}
```

1.2 База 1 - вектор

```
char sign(dbl x) { return (x > EPS) - (x < -EPS); }
struct vctr {
  dbl x, y;
  vctr() {}
  vctr(dbl x, dbl y) : x(x), y(y) {}
  dbl operator%(const vctr &o)const{return x*o.x+y*o.y;}
  dbl operator*(const vctr &o)const{return x*o.y-y*o.x;}
  vctr operator+(const vctr &o)const{return{x+o.x,y+o.y};}
  vctr operator-(const vctr &o)const{return{x-o.x,y-o.y};}
  vctr operator-() const { return {-x, -y}; }
  vctr operator*(const dbl d)const{return{x*d,y*d};}
  vctr operator/(const dbl d)const{return{x/d,y/d};}
  void operator+=(const vctr &o) { x += o.x, y += o.y; }
  void operator-=(const vctr &o) { x -= o.x, y -= o.y; }
  bool operator==(const vctr &o)const{ return !sign(x-o.x) &&
     !sign(y-o.y); }
  dbl dist2() const { return x * x + y * y; }
  dbl dist() const { return sqrtl(dist2()); }
  vctr norm() const { return *this / dist(); }
  vctr rotate_ccw_90() const {  return {-y, x}; }
};

// противоположные (a*b == 0): +pi или -pi, знак от шума
dbl angle_between(const vctr &a, const vctr &b) {
  return atan2(a*b, a%b); // ccw от a до b, знак совпадает с a
     *b
}

const dbl PI = acosl(-1);

// ccw от a до b в [0, 2pi), ~2pi -> 0
dbl angle_between_positive(const vctr &a, const vctr &b) {
  dbl r = atan2(a*b, a%b);
  if (r < 0) r += 2 * PI;
  return r > 2 * PI - EPS ? 0 : r;
}

// y > 0 ? 0 : 1
bool is2plane(const vctr &a) {
  return sign(a.y)<0 || (sign(a.y)==0 && sign(a.x)<0);
}

bool cmp_angle(const vctr &a, const vctr &b) {
  bool pla = is2plane(a);
  bool plb = is2plane(b);
  if (pla != plb) return pla < plb;
  return sign(a * b) > 0;
}

vctr rotate_ccw(const vctr &a, dbl phi) {
  dbl cs = cosl(phi);
  dbl sn = sinl(phi);
  return {a.x * cs - a.y * sn, a.y * cs + a.x * sn};
}
```

1.3 База 2 - прямая

```
struct line {
  dbl a, b, c;
  line() {}
  line(dbl a, dbl b, dbl c) : a(a), b(b), c(c) {}
  line(const vctr A, const vctr B) {
    a = A.y - B.y;
    b = B.x - A.x;
    c = A * B;
    // left halfplane of A->B is positive
    // assert(a != 0 || b != 0);
  }
  void operator*=(dbl x) { a *= x, b *= x, c *= x; }
  void operator/=(dbl x) { a /= x, b /= x, c /= x; }
  dbl get(const vctr P) const { return a * P.x + b * P.y + c;
     }
  vctr anyPoint() const {
    dbl x = -a * c / (a * a + b * b);
    dbl y = -b * c / (a * a + b * b);
    return {x, y};
  }
  void normalize() {
    dbl d = sqrtl(a * a + b * b);
    a /= d, b /= d, c /= d;
  }
};

bool isparallel(line l1, line l2) {
  return sign(l2.a * l1.b - l2.b * l1.a) == 0;
}

vctr intersection(const line &l1, const line &l2) {
  dbl z = l2.a * l1.b - l2.b * l1.a;
  dbl x = (l1.c * l2.b - l2.c * l1.b) / z;
  dbl y = -(l1.c * l2.a - l2.c * l1.a) / z;
  return {x, y};
}

// Серединный перпендикуляр (не биссектриса!)
line bisection(const vctr A, const vctr B) {
```

```
  vctr M = (A + B) / 2;
  return line(M, M + (B - A).rotate_ccw_90());
}
```

1.4 База 3 - окружность

```
struct circle {
  vctr C;
  dbl r;
  circle() {}
  circle(dbl x, dbl y, dbl r) : C(x, y), r(r) {}
  circle(vctr C, dbl r) : C(C), r(r) {}
  circle(const vctr A, const vctr B) {
    C = (A + B) / 2;
    r = (A - B).dist() / 2;
  }
  circle(const vctr A, const vctr B, const vctr D) {
    vctr u = B-A, v = D-A;
    dbl p = u.dist2(), q = v.dist2();
    C = A + vctr(v.y*p - u.y*q, u.x*q - v.x*p) / (2*(u*v));
    r = (C - A).dist();
  }
  bool isin(const vctr P, char strict=0) const {
    // `dist2 - r*r` fails on dbl precision.
    return sign((C - P).dist() - r) <= -strict;
    // return sign((C - P).dist2() - r * r) <= -strict;
  }
};

vector<vctr> intersection_line_circ(line l, circle c) {
  l.normalize();
  dbl s = l.get(c.C), d = abs(s);
  if (sign(d - c.r) > 0) return {};
  vctr a = c.C - vctr(l.a, l.b) * s;
  if (sign(c.r - d) == 0) return {a};
  dbl k = sqrtl((c.r - d) * (c.r + d));
  vctr v = vctr(l.b, -l.a) * k;
  // arc from res[0] to res[1] ccw lies in positive halfplane
  return {a + v, a - v};
}

vector<vctr> intersection_circ_circ(circle A, circle B) {
  vctr d = B.C - A.C; // better precision
  line l(2*d.x, 2*d.y,
         (B.r-A.r) * (B.r+A.r) - d.dist2());
  if (sign(l.a) == 0 && sign(l.b) == 0) return {};
  auto c = A.r<B.r ? circle(0,0,A.r) : circle(d,B.r); //
     better precision
  auto res = intersection_line_circ(l, c);
  for (auto &p : res) p += A.C;
  // arc from res[0] to res[1] ccw over A is inside B
  // <=> ccw over B is outside A
  return res;
}

vector<vctr> tangent_vctr_circ(vctr v, circle c) {
  dbl d = (c.C - v).dist();
  if (sign(d - c.r) < 0) return {};
  if (sign(d - c.r) == 0) return {v};
  dbl k = sqrtl((d - c.r) * (d + c.r));
  // res[0] is right tangent, res[1] is left tangent
  return intersection_circ_circ(circle(v.x, v.y, k), c);
}

// {point on A, point on B}: outer tangents, then inner.
// 4 apart, 3 touch outside, 2 intersect, 1 touch inside,
// 0 nested. Tangent at touching point has equal points
// (line is perpendicular to A.C->B.C)
// Returns (looking from A.C to B.C):
// {left, left}, {right, right}, {left, right}, {right, left}
vector<pair<vctr, vctr>> common_tangents(circle A, circle B) {
  vctr d = B.C - A.C;
  dbl d2 = d.dist2();
  if (sign(d2) == 0) return {};
  vector<pair<vctr, vctr>> res;
  for (dbl r : {B.r, -B.r}) { // outer, inner
    dbl dr = A.r - r, h2 = d2 - dr * dr;
    if (sign(h2) < 0) continue;
    dbl h = sqrtl(max<dbl>(h2, 0));
    for (int s : {1, -1}) {
      vctr v = (d * dr + d.rotate_ccw_90() * (s * h)) / d2;
      res.pb({A.C + v * A.r, B.C + v * r});
      if (sign(h2) == 0) break;
    }
  }
  return res;
}
```

1.5 Выпуклая оболочка

```
// need_all: also points on edges
vector<vctr> convex_hull(vector<vctr> p, bool need_all = 0) {
  sort(all(p), [](const vctr &a, const vctr &b) {
    return tie(a.y, a.x) < tie(b.y, b.x);
  });
  p.erase(unique(all(p)), end(p)); // must have
  int n = p.size(), k = 0;
  if (n <= 2) return p;
  auto bad = [&](vctr a, vctr b, vctr c) { // pop b?
    return sign((b - a) * (c - b)) < !need_all;
  };
  vector<vctr> ch(2 * n);
  for (int i = 0; i < n; ch[k++] = p[i++])
    while (k >= 2 && bad(ch[k-2], ch[k-1], p[i])) --k;
  for (int i = n - 2, t = k + 1; i >= 0; ch[k++] = p[i--])
    while (k >= t && bad(ch[k-2], ch[k-1], p[i])) --k;
  ch.resize(k - 1);
// If hull is seg and need_all: p[0] p[1] .. p[n-1] .. p[1].
// If need only one entry in this case: ch.resize(min(k-1, n))
    ;
  return ch;
}
```

1.6 Задача 16

```
bool isInSameHalf(vctr p, vctr r1, vctr r2) {
  return sign((r2 - r1) % (p - r1)) >= 0;
}

dbl distPointPoint(vctr a, vctr b) {
  return (a - b).dist();
}

dbl distPointLine(vctr a, vctr l1, vctr l2) {
  line l(l1, l2);
  l.normalize();
  return abs(l.get(a));
}

dbl distPointRay(vctr a, vctr r1, vctr r2) {
  if (r1 == r2 || !isInSameHalf(a, r1, r2))
    return distPointPoint(a, r1);
  return distPointLine(a, r1, r2);
}

dbl distPointSeg(vctr a, vctr s1, vctr s2) {
  return max(distPointRay(a, s1, s2),
             distPointRay(a, s2, s1));
}

bool isIntersectionLineLine(line l1, line l2) {
  dbl znam = l1.b * l2.a - l1.a * l2.b;
  return sign(znam) != 0;
}

vctr intersectionLineLine(line l1, line l2) {
  dbl znam = l1.b * l2.a - l1.a * l2.b;
  dbl y = -(l1.c * l2.a - l2.c * l1.a) / znam;
  dbl x = -(l1.c * l2.b - l2.c * l1.b) / -znam;
  return vctr(x, y);
}

dbl distLineLine(vctr l1a, vctr l1b, vctr l2a, vctr l2b) {
  line l1(l1a, l1b);
  line l2(l2a, l2b);
  if (isIntersectionLineLine(l1, l2))
    return 0;
  return distPointLine(l1a, l2a, l2b);
}

dbl distRayLine(vctr r1, vctr r2, vctr l1, vctr l2) {
  if (r1 == r2) return distPointLine(r1, l1, l2);
  line r(r1, r2);
  line l(l1, l2);
  if (!isIntersectionLineLine(l, r))
    return distLineLine(r1, r2, l1, l2);
  vctr p = intersectionLineLine(l, r);
  if (isInSameHalf(p, r1, r2))
    return 0;
  return distPointLine(r1, l1, l2);
}

dbl distSegLine(vctr s1, vctr s2, vctr l1, vctr l2) {
  return max(distRayLine(s1, s2, l1, l2),
             distRayLine(s2, s1, l1, l2));
}

dbl distRayRay(vctr r1a, vctr r1b, vctr r2a, vctr r2b) {
  if (r1a == r1b) return distPointRay(r1a, r2a, r2b);
  if (r2a == r2b) return distPointRay(r2a, r1a, r1b);
  line r1(r1a, r1b);
  line r2(r2a, r2b);
  if (!isIntersectionLineLine(r1, r2)) {
    if (isInSameHalf(r1a, r2a, r2b) || isInSameHalf(r2a, r1a,
     r1b))
      return distLineLine(r1a, r1b, r2a, r2b);
    else
      return distPointPoint(r1a, r2a);
  }
  vctr p = intersectionLineLine(r1, r2);
  if (isInSameHalf(p, r1a, r1b) && isInSameHalf(p, r2a, r2b))
    return 0;
  return min(distPointRay(r1a, r2a, r2b),
             distPointRay(r2a, r1a, r1b));
}

dbl distSegRay(vctr s1, vctr s2, vctr r1, vctr r2) {
  return max(distRayRay(s1, s2, r1, r2),
```

```
            distRayRay(s2, s1, r1, r2));
}

dbl distSegSeg(vctr s1a, vctr s1b, vctr s2a, vctr s2b) {
  return max(distSegRay(s1a, s1b, s2a, s2b),
             distSegRay(s1a, s1b, s2b, s2a));
}
```

1.7 Калиперы

```
// Диаметр выпуклого многоугольника
dbl calipers(vector<vctr> &pts) {
  int n = pts.size();
  int a = 0, b = 0;
  for (int i = 1; i < n; ++i) {
    auto &v = pts[i];
    if (tie(v.y, v.x) < tie(pts[a].y, pts[a].x))
      a = i;
    if (tie(v.y, v.x) > tie(pts[b].y, pts[b].x))
      b = i;
  }
  int aa = (a + 1) % n, bb = (b + 1) % n;
  dbl dist2 = 0;
  for (int i = 0; i < n; ++i) {
    while (sign((pts[aa] - pts[a]) * (pts[bb] - pts[b])) > 0)
      b = bb, bb = (b + 1) % n;
    dist2 = max(dist2, (pts[a] - pts[b]).dist2());
    a = aa, aa = (a + 1) % n;
  }
  return sqrtl(dist2);
}
```

1.8 Касательные к двум многоугольникам

```
// A, B convex (CCW or CW, independently), no 3 collinear,
// no common points.                          O(log n log m).
// Returns (looking from A[i] to B[j]):
// {left, left}, {left, right}, {right, left}, {right, right}
vector<pair<int, int>> tangents_polys(vector<vctr> &A,
                                      vector<vctr> &B) {
  int n = A.size(), logn = 31 - __builtin_clz(n);
  vector<pair<int, int>> res;
  for (int sa : {1, -1}) for (int sb : {1, -1}) {
    auto t = [&](int i) {
      auto [r, l] = tangents_from_point(A[i], B);
      return sb > 0 ? r : l;
    };
    auto d = [&](int i) { return B[t(i)] - A[i]; };
    int i = 0;
    for (int k = logn; k >= 0; --k) {
      int i1 = (i - (1 << k) + n) % n;
      int i2 = (i + (1 << k)) % n;
      if (sign(d(i) * d(i1)) == sa) i = i1;
      if (sign(d(i) * d(i2)) == sa) i = i2;
    }
    res.pb({i, t(i)});
  }
  return res;
}
```

1.9 Касательные к многоугольнику

```
// p convex (CCW or CW), no 3 collinear, a outside.
// {right, left} tangents looking from a to p
pair<int, int> tangents_from_point(vctr &a, vector<vctr> &p) {
  int n = p.size();
  int logn = 31 - __builtin_clz(n);
  auto findWithSign = [&](int sgn) {
    int i = 0;
    for (int k = logn; k >= 0; --k) {
      int i1 = (i - (1 << k) + n) % n;
      int i2 = (i + (1 << k)) % n;
      if (sign((p[i1] - a) * (p[i] - a)) == sgn) i = i1;
      if (sign((p[i2] - a) * (p[i] - a)) == sgn) i = i2;
    }
    return i;
  };
  return {findWithSign(1), findWithSign(-1)};
}

// p convex, no 3 collinear. Point with max (sgn=1) or
// min (sgn=-1) signed distance to line
int tangent_parallel_line(const vector<vctr> &p, line l, int
    sgn) {
  l *= sgn;
  int n = p.size();
  int i = 0;
  int logn = 31 - __builtin_clz(n);
  for (int k = logn; k >= 0; --k) {
    int i1 = (i - (1 << k) + n) % n;
    int i2 = (i + (1 << k)) % n;
    if (l.get(p[i1]) > l.get(p[i])) i = i1;
    if (l.get(p[i2]) > l.get(p[i])) i = i2;
  }
  return i;
}
```

1.10 Лежит ли точка в многоугольнике

```
// convex CCW, no 3 collinear, any start, size >= 3
bool is_point_in_convex_poly(vctr a, const vector<vctr> &p,
                             bool strict = 0) {
  int n = p.size(), l=1, r=n-1;
  auto s = [&](int i, int j) {
    return sign((p[j] - p[i]) * (a - p[i]));
  };
  if (s(0, 1) < strict || s(0, n-1) > -strict) return false;
  while (r-l>1) {
    int m = (l+r)/2;
    (s(0, m) >= 0 ? l : r) = m;
  }
  return s(l, l+1) >= strict;
}

// p: any simple polygon, any orientation. O(n)
bool is_point_in_any_poly(vctr a, const vector<vctr> &p,
                          bool strict = 0) {
  int n = p.size(), cnt = 0;
  for (int i = 0, j = n - 1; i < n; j = i++) {
    vctr u = p[j] - a, v = p[i] - a;
    if (!sign(u * v) && sign(u % v) <= 0) return !strict;
    cnt ^= ((0 < u.y) - (0 < v.y)) * sign(u * v) > 0;
  }
  return cnt;
}
```

1.11 Минимальная покрывающая окружность

```
circle MinDisk2(vector<vctr> &p, vctr A, vctr B, int sz) {
  circle w(A, B);
  for (int i = 0; i < sz; ++i) {
    if (w.isin(p[i])) continue;
    w = circle(A, B, p[i]);
  }
  return w;
}
circle MinDisk1(vector<vctr> &p, vctr A, int sz) {
  circle w(A, p[0]);
  for (int i = 1; i < sz; ++i) {
    if (w.isin(p[i])) continue;
    w = MinDisk2(p, A, p[i], i);
  }
  return w;
}
circle MinDisk(vector<vctr> &p) {
  int sz = p.size();
  if (sz == 1) return circle(p[0], 0);
  shuffle(p.begin(), p.end(), rnd);
  circle w(p[0], p[1]);
  for (int i = 2; i < sz; ++i) {
    if (w.isin(p[i])) continue;
    w = MinDisk1(p, p[i], i);
  }
  return w;
}
```

1.12 Пересечение полуплоскостей

```
// For int line let N = max(|a|,|b|), K = max|c|, then:
// O(log) 'compare' needs 2N^2 and 2NK fit into type;
// cross-multiplication 'compare' needs 4N^3K.


// Half plane: ax+by+c >= 0. Bounding box MUST HAVE.
vector<int> calc_hpi_inds(const vector<line> &ls) {
  int n = ls.size(), m=0, l=0, r=0;
  vector<int> q(n);
  vector<char> h(n);
  iota(q.begin(), q.end(), 0);
  auto cr = [&](int i, int j) {
    return ls[i].a*ls[j].b-ls[i].b*ls[j].a;
  };
  auto bigb = [&](int i){return abs(ls[i].b)>=abs(ls[i].a);};
  auto num = [&](int i, int j) {
    if (bigb(j)) return ls[i].b*ls[j].c-ls[i].c*ls[j].b;
    return ls[i].c*ls[j].a-ls[i].a*ls[j].c;
  };
  auto dir = [&](int i) {
    return bigb(i) ? sign(ls[i].b) : -sign(ls[i].a);
  };
  auto on = [&](int i, int j){return -sign(num(i,j))*dir(j);};
  auto at = [&](int i, int j, int k) {
    // return (int)sign(ls[k].get(intersection(ls[i],ls[j])));
    // uncomment above and comment code below if float line
    if (!cr(j,k)) return on(k,j);
    return -sign(cr(j,k))*dir(j)*compare(
      num(i,j),cr(i,j),num(k,j),cr(k,j));
  };
  auto same_dir = [&](int i, int j) {
    return !sign(cr(i,j))&&ls[i].a*ls[j].a+ls[i].b*ls[j].b>0;
  };
```

```
  for (int i = 0; i < n; ++i)
    h[i] = sign(ls[i].b)<0||(!sign(ls[i].b)&&ls[i].a<0);
  sort(q.begin(), q.end(), [&](int i, int j) { // 70% of time
    return h[i] != h[j] ? h[i] < h[j] : cr(i,j) > 0;
  });
  for (int i : q) {
    if (!m || !same_dir(q[m-1],i)) q[m++] = i;
    else if (on(q[m-1],i) >= 0) q[m-1] = i;
  }

  auto bad = [&](int i, int j, int k) {
    if (!sign(cr(i,j))) return false;
    int s = at(i,j,k), t = sign(cr(i,k));
    return s<0 || (!s&&t&&sign(cr(i,j))==t&&sign(cr(j,k))==t);
  };
  for (int t = 0; t < m; ++t) {
    int i = q[t];
    while (r-l>1 && bad(q[r-2],q[r-1],i)) --r;
    while (r-l>1 && bad(q[l+1],q[l],i)) ++l;
    if (l<r && same_dir(q[r-1],i) && on(q[r-1],i)<0)
      return {};
    q[r++] = i;
  }
  while (r-l>2 && bad(q[r-2],q[r-1],q[l])) --r;
  while (r-l>2 && bad(q[l+1],q[l],q[r-1])) ++l;
  if (r-l < 3) return {};
  return {q.begin()+l,q.begin()+r};
}
// below 'vctr' must be float even if 'line' is integer
vector<vctr> half_plane_intersection(const vector<line> &ls) {
  vector<int> inds = calc_hpi_inds(ls);
  int n = inds.size();
  vector<vctr> pts;
  pts.reserve(n);
  for (int i = 0; i < n; ++i) {
    vctr P = intersection(ls[inds[i]],ls[inds[(i+1)%n]]);
    if (pts.empty() || !(pts.back() == P)) pts.pb(P);
  }
  if (pts.size()>1 && pts[0] == pts.back()) pts.pop_back();
// "empty"->{}, "point"->{P}, "segment"->{A,B}, else CCW.
// 'inds' have no lines with 0 length, except for "segment"
// (4 lines), and "point" (3 lines, or 4 if 2 and 2 parallel)
  return pts;
}
```

1.13 Проверка на пересечение отрезков

```
bool is_intersection_seg(vctr A, vctr B, vctr C, vctr D) {
  for (int i = 0; i < 2; ++i) {
    auto l1 = A.x, r1 = B.x, l2 = C.x, r2 = D.x;
    if (l1 > r1) swap(l1, r1);
    if (l2 > r2) swap(l2, r2);
    if (max(l1, l2) > min(r1, r2))
      return false;
    swap(A.x, A.y);
    swap(B.x, B.y);
    swap(C.x, C.y);
    swap(D.x, D.y);
  }
  for (int _ = 0; _ < 2; ++_) {
    auto v1 = (B - A) * (C - A);
    auto v2 = (B - A) * (D - A);
    if (sign(v1) * sign(v2) == 1)
      return false;
    swap(A, C);
    swap(B, D);
  }
  return true;
}
```

1.14 Сумма Минковского

```
// A, B: CCW from min (y, x), no 3 collinear, size >= 1.
// Result: CCW from min, no points on sides, no duplicates
vector<vctr> minkowski_sum(vector<vctr> A, vector<vctr> B) {
  int n = A.size(), m = B.size();
  A.pb(A[0]), A.pb(A[1 % n]);
  B.pb(B[0]), B.pb(B[1 % m]);
  vector<vctr> C; C.reserve(n+m);
  for (int i = 0, j = 0; i < n || j < m; ) {
    C.pb(A[i] + B[j]);
    vctr a = A[i + 1] - A[i], b = B[j + 1] - B[j];
    // cmp_angle from vctr template
    int c = j==m ? 1 : i==n || cmp_angle(b,a) ? -1 : cmp_angle
     (a,b);
    if (c >= 0) ++i;
    if (c <= 0) ++j;
  }
  return C;
}
```

2 Графы

2.1 2-SAT

```
struct TwoSat {
  int n;
  vector<vector<int>> g, rg;
  vector<int> comp, topsort;
  vector<char> used;

  TwoSat(int n) : n(n) {
    g.resize(2 * n);
    rg.resize(2 * n);
    topsort.reserve(2 * n);
  }

  int neg(int v) {
    return 2 * n - 1 - v;
  }

  void add(int v, int u) {
    g[v].pb(u);
    rg[u].pb(v);
  }

  void add_OR(int v, int u) { // v | u
    add(neg(v), u), add(neg(u), v);
  }

  void add_IMPL(int v, int u) { // v -> u
    add_OR(neg(v), u);
  }

  void dfs1(int v) {
    used[v] = 1;
    for (auto u: g[v]) {
      if (!used[u])
        dfs1(u);
    }
    topsort.push_back(v);
  }

  void dfs2(int v, int col) {
    comp[v] = col;
    for (auto u: rg[v]) {
      if (comp[u] == -1)
        dfs2(u, col);
    }
  }

  void SCC() {
    used.assign(2 * n, 0), comp.assign(2 * n, -1);
    topsort.clear();
    for (int v = 0; v < 2 * n; ++v) {
      if (!used[v])
        dfs1(v);
    }
    reverse(all(topsort));
    int cc = 0;
    for (int i = 0; i < 2 * n; ++i) {
      if (comp[topsort[i]] == -1)
        dfs2(topsort[i], cc++);
    }
  }

  vector<char> solution() {
    assert(n > 0); // returns {} if ans exists AND not
    SCC();
    vector<char> ans(n);
    for (int v = 0; v < n; ++v) {
      if (comp[v] == comp[neg(v)])
        return {}; // no solution
      ans[v] = comp[v] > comp[neg(v)];
    }
    return ans;
  }
};
```

2.2 L-R Flow

```
struct LRFlow {
  Dinic dinic; // ориентированные рёбра
  int S, T; // исток и сток
  int Sx, Tx; // вспомогательные вершины, любые неиспользуемые
      индексы
  vector<ll> dem;

  LRFlow(int n, int S, int T, int Sx, int Tx)
    : dinic(n), S(S), T(T), Sx(Sx), Tx(Tx), dem(n) {}

  // returns id: real flow = dinic.graph[v][id].f + mincap
  int addedge(int v, int u, flow_t mincap, flow_t maxcap) {
    dem[v] -= mincap, dem[u] += mincap;
    return dinic.addedge(v, u, maxcap - mincap);
  }

  // returns true if found flow is feasible.
  // size of flow (after aux edges are removed below):
  // sum e.f over dinic.graph[S] - dem[S]
  bool run() {
    ll totaldem = 0;
    for (int v = 0; v < dem.size(); ++v) {
      if (dem[v] > 0) {
        totaldem += dem[v];
```

```
      // aux capacities are sums: flow_t must fit totaldem
    (~ sum of mincaps)
      dinic.addedge(Sx, v, dem[v]);
    } else if (dem[v] < 0)
      dinic.addedge(v, Tx, -dem[v]);
  }
  dinic.addedge(T, S, totaldem); // totaldem is enough
  dinic.addedge(S, T, totaldem); // for total flow < 0
  dinic.run(Sx, Tx);
  ll flow = 0;
  for (auto &e : dinic.graph[Sx]) flow += e.f;
  if (flow != totaldem) return false;
  // return true; // if want ONLY feasibility
  for (int v = 0; v < dem.size(); ++v) {
    if (dem[v]) dinic.graph[v].pop_back();
  }
  dinic.graph[Sx].clear();
  dinic.graph[Tx].clear();
  for (int i = 0; i < 2; ++i) // T->S, S->T
    dinic.graph[S].pop_back(), dinic.graph[T].pop_back();
  dinic.run(S, T); // if we want max lr-flow (not any lr-
   flow)
  return true;
 }
};
```

2.3 WeightedMatching

```
// НЕ ЗАБЫТЬ вызвать init(n)
// Вершины нумеруются от 1 до n. Петли и кратные рёбра - ок.
// Ищет паросочетание max веса, число рёбер любое.
namespace weighted_matching{
  const ll INF = 1e18; // > 4 * max w
  const int MAXN = 1050; //double of possible N
  struct E { int x, y; ll w; };
  int n, m;
  E G[MAXN][MAXN];
  ll lab[MAXN];
  int match[MAXN], slack[MAXN], st[MAXN], pa[MAXN];
  int flo_from[MAXN][MAXN], S[MAXN], vis[MAXN];
  vector<int> flo[MAXN];
  queue<int> Q;
  void init(int _n) {
    n = _n;
    for(int x = 1; x <= n; ++x)
      for(int y = 1; y <= n; ++y)
        G[x][y] = E{x, y, 0};
  }
  void add_edge(int x, int y, ll w) {
    G[x][y].w = G[y][x].w = max(G[x][y].w, w);
  }
  ll e_delta(E e) {
    return lab[e.x] + lab[e.y] - G[e.x][e.y].w * 2;
  }
  void update_slack(int u, int x) {
    if(!slack[x] || e_delta(G[u][x]) < e_delta(G[slack[x]][x])
     )
      slack[x] = u;
  }
  void set_slack(int x) {
    slack[x] = 0;
    for(int u = 1; u <= n; ++u)
      if(G[u][x].w > 0 && st[u] != x && S[st[u]] == 0)
        update_slack(u, x);
  }
  void q_push(int x) {
    if(x <= n) Q.push(x);
    else for(int i = 0; i < (int)flo[x].size(); ++i)
        q_push(flo[x][i]);
  }
  void set_st(int x, int b) {
    st[x] = b;
    if(x > n) for(int i = 0; i < (int)flo[x].size(); ++i)
        set_st(flo[x][i], b);
  }
  int get_pr(int b, int xr) {
    int pr = find(flo[b].begin(), flo[b].end(), xr) - flo[b].
     begin();
    if(pr & 1) {
      reverse(flo[b].begin() + 1, flo[b].end());
      return (int)flo[b].size() - pr;
    }
    else return pr;
  }
  void set_match(int x, int y) {
    match[x] = G[x][y].y;
    if(x <= n) return;
    E e = G[x][y];
    int xr = flo_from[x][e.x], pr = get_pr(x, xr);
    for(int i = 0; i < pr; ++i) set_match(flo[x][i], flo[x][i
     ^1]);
    set_match(xr, y);
    rotate(flo[x].begin(), flo[x].begin() + pr, flo[x].end());
  }
  void augment(int x, int y) {
    while(1) {
      int ny = st[match[x]];
      set_match(x, y);
      if(!ny) return;
      set_match(ny, st[pa[ny]]);
      x = st[pa[ny]], y = ny;
    }
  }
  int get_lca(int x, int y) {
    static int t = 0;
    for(++t; x || y; swap(x, y)) {
      if(x == 0) continue;
      if(vis[x] == t) return x;
      vis[x] = t;
      x = st[match[x]];
      if(x) x = st[pa[x]];
    }
    return 0;
  }
  void add_blossom(int x, int l, int y) {
    int b = n + 1;
    while(b <= m && st[b]) ++b;
    if(b > m) ++m;
    lab[b] = 0, S[b] = 0;
    match[b] = match[l];
    flo[b].clear();
    flo[b].pb(l);
    for(int u = x, v; u != l; u = st[pa[v]])
      flo[b].pb(u), flo[b].pb(v = st[match[u]]), q_push(v);
    reverse(flo[b].begin() + 1, flo[b].end());
    for(int u = y, v; u != l; u = st[pa[v]])
      flo[b].pb(u), flo[b].pb(v = st[match[u]]), q_push(v);
    set_st(b, b);
    for(int i = 1; i <= m; ++i) G[b][i].w = G[i][b].w = 0;
    for(int i = 1; i <= n; ++i) flo_from[b][i] = 0;
    for(int i = 0; i < (int)flo[b].size(); ++i) {
      int us = flo[b][i];
      for(int u = 1; u <= m; ++u)
        if(G[b][u].w == 0 || e_delta(G[us][u]) < e_delta(G[b][
     u]))
          G[b][u] = G[us][u], G[u][b] = G[u][us];
      for(int u = 1; u <= n; ++u)
        if(flo_from[us][u])
          flo_from[b][u] = us;
    }
    set_slack(b);
  }
  void expand_blossom(int b) {
    for(int i = 0; i < (int)flo[b].size(); ++i)
      set_st(flo[b][i], flo[b][i]);
    int xr = flo_from[b][G[b][pa[b]].x], pr = get_pr(b, xr);
    for(int i = 0; i < pr; i += 2) {
      int xs = flo[b][i], xns = flo[b][i + 1];
      pa[xs] = G[xns][xs].x;
      S[xs] = 1, S[xns] = 0;
      slack[xs] = 0, set_slack(xns);
      q_push(xns);
    }
    S[xr] = 1, pa[xr] = pa[b];
    for(int i = pr + 1; i < (int)flo[b].size(); ++i) {
      int xs = flo[b][i];
      S[xs] = -1, set_slack(xs);
    }
    st[b] = 0;
  }
  bool on_found_edge(E e) {
    int x = st[e.x], y = st[e.y];
    if(S[y] == -1) {
      pa[y] = e.x, S[y] = 1;
      int ny = st[match[y]];
      slack[y] = slack[ny] = 0;
      S[ny] = 0, q_push(ny);
    }
    else if(S[y] == 0) {
      int l = get_lca(x, y);
      if(!l) return augment(x, y), augment(y, x), true;
      else add_blossom(x, l, y);
    }
    return false;
  }
  bool matching() {
    fill(S + 1, S + m + 1, -1);
    fill(slack + 1, slack + m + 1, 0);
    Q = queue<int>();
    for(int x = 1; x <= m; ++x)
      if(st[x] == x && !match[x]) pa[x] = 0, S[x] = 0, q_push(
     x);
    if(Q.empty()) return false;
    while(1) {
      while(Q.size()) {
        int x = Q.front(); Q.pop();
        if(S[st[x]] == 1) continue;
        for(int y = 1; y <= n; ++y) {
          if(G[x][y].w > 0 && st[x] != st[y]) {
            if(e_delta(G[x][y]) == 0) {
              if(on_found_edge(G[x][y])) return true;
            }
            else update_slack(x, st[y]);
          }
        }
      }
      ll d = INF;
      for(int b = n + 1; b <= m; ++b)
        if(st[b] == b && S[b] == 1) d = min(d, lab[b] / 2);
      for(int x = 1; x <= m; ++x)
        if(st[x] == x && slack[x]) {
```

```
        if(S[x] == -1) d = min(d, e_delta(G[slack[x]][x]));
        else if(S[x] == 0) d = min(d, e_delta(G[slack[x]][x
    ]) / 2);
      }
    for(int x = 1; x <= n; ++x) {
      if(S[st[x]] == 0) {
        if(lab[x] <= d) return 0;
        lab[x] -= d;
      }
      else if(S[st[x]] == 1) lab[x] += d;
    }
    for(int b = n + 1; b <= m; ++b)
      if(st[b] == b) {
        if(S[st[b]] == 0) lab[b] += d * 2;
        else if(S[st[b]] == 1) lab[b] -= d * 2;
      }
    Q = queue<int>();
    for(int x = 1; x <= m; ++x)
      if(st[x] == x && slack[x] && st[slack[x]] != x &&
   e_delta(G[slack[x]][x]) == 0)
        if(on_found_edge(G[slack[x]][x])) return true;
    for(int b = n + 1; b <= m; ++b)
      if(st[b] == b && S[b] == 1 && lab[b] == 0)
        expand_blossom(b);
  }
  return false;
 }
 pair<ll, int> solve(vector<pair<int, int>> &ans) {
  fill(match + 1, match + n + 1, 0);
  m = n;
  int cnt = 0; ll sum = 0;
  for(int u = 0; u <= n; ++u) st[u] = u, flo[u].clear();
  ll mx = 0;
  for(int x = 1; x <= n; ++x)
    for(int y = 1; y <= n; ++y){
      flo_from[x][y] = (x == y ? x : 0);
      mx = max(mx, G[x][y].w);
    }
  for(int x = 1; x <= n; ++x) lab[x] = mx;
  while(matching()) ++cnt;
  for(int x = 1; x <= n; ++x)
    if(match[x] && match[x] < x) {
      sum += G[x][match[x]].w;
      ans.pb({x, G[x][match[x]].y});
    }
  return {sum, cnt};
 }
}
// n вершин, W = max|w|. давать w':
//   max вес, затем max рёбер: w' = n * w + 1
//   max вес, затем min рёбер: w' = n * w - 1
//   min вес, затем ...: то же с -w вместо w
//   max рёбер, затем max вес: w' = n * W + 1 + w
//   max рёбер, затем min вес: w' = n * W + 1 - w
// вес ответа считать по исходным w. нужно 4 * max w' < INF
```

2.4 Венгерский алгоритм

```
pair<ll, vector<int>> venger(vector<vector<ll>> a) {
// ищет минимальное по стоимости
// работает только при n <= m
// a - массив весов (n+1) x (m+1)
// a[0][..] = a[..][0] = 0
// a[i][j] любые (>0, <0, =0), n * max|a[i][j]| <= 1.5e18
// возвращает ans[i] = j если взяли ребро a[i][j]
  int n = ssize(a) - 1;
  int m = ssize(a[0]) - 1;
  vector<ll> u(n + 1), v(m + 1);
  vector<int> p(m + 1), way(m + 1);
  const ll INF = LLONG_MAX;
  for (int i = 1; i <= n; ++i) {
    p[0] = i;
    int y = 0;
    vector<ll> minv(m + 1, INF);
    vector<char> used(m + 1, false);
    do {
      used[y] = true;
      int x = p[y], z;
      ll delta = INF;
      for (int j = 1; j <= m; ++j) {
        if (!used[j]) {
          ll cur = a[x][j] - u[x] - v[j];
          if (cur < minv[j])
            minv[j] = cur, way[j] = y;
          if (minv[j] < delta)
            delta = minv[j], z = j;
        }
      }
      for (int j = 0; j <= m; ++j) {
        if (used[j])
          u[p[j]] += delta, v[j] -= delta;
        else
          minv[j] -= delta;
      }
      y = z;
    } while (p[y] != 0);
    do {
      int z = way[y];
      p[y] = p[z];
      y = z;
    } while (y);
  }
  ll cost = -v[0];
  vector<int> ans(n + 1);
  for (int j = 1; j <= m; ++j)
    ans[p[j]] = j;
  return {cost, ans};
}
```

2.5 Вершинная двусвязность

```
vector<pair<int, int>> graph[MAX_V];
bitset<MAX_V> vis;
int st[MAX_E], col[MAX_E], tin[MAX_V], up[MAX_V];
int sti = 0, cc = 0, tt = 0;

void dfs(int v, int pei) {
  vis[v] = true;
  int upv = tin[v] = tt++;
  for (auto [u, ei] : graph[v]) {
    if (ei == pei) continue;
    if (!vis[u]) {
      int pt = sti;
      st[sti++] = ei;
      dfs(u, ei);
      upv = min(upv, up[u]);
      if (up[u] >= tin[v]) {
        while (sti > pt)
          col[st[--sti]] = cc;
        cc++;
      }
    } else if (tin[u] < tin[v]) {
      st[sti++] = ei;
      upv = min(upv, tin[u]);
    }
  }
  up[v] = upv;
}

// graph[v].emplace_back(u, i);
// graph[u].emplace_back(v, i);
fill(col, col + m, -1);
for (int v = 0; v < n; ++v) {
  if (!vis[v])
    dfs(v, -1);
}
// col[i] - компонента i-го ребра (у петель -1)
// cc - итоговое кол-во компонент
```

2.6 Диниц

```
// flow_t: capacity of one edge, total flow is ll
// undirected edges: flow_t must fit 2c
using flow_t = int;
const int LOG = 29; // масштабирование, =0 если не нужно
struct Edge { int u, r; flow_t f, c; };

struct Dinic {
  vector<vector<Edge>> graph;
  vector<char> vis;
  vector<int> inds, dist, Q;
  int ql, qr, S, T;
  flow_t BIT; // 2^LOG must fit flow_t
  Dinic(int n) : graph(n), vis(n), inds(n), dist(n), Q(n) {}

  bool bfs() {
    memset(vis.data(), 0, vis.size());
    ql = 0, qr = 0;
    dist[S] = 0, vis[S] = true;
    Q[qr++] = S;
    while (ql < qr) {
      int v = Q[ql++];
      for (auto &e : graph[v]) {
        int u = e.u;
        if (vis[u] || e.c - e.f < BIT)
          continue;
        vis[u] = true;
        dist[u] = dist[v] + 1;
        Q[qr++] = u;
        if (u == T) return true;
      }
    }
    return false;
  }

  flow_t dfs(int v, flow_t maxF) {
    if (v == T) return maxF;
    flow_t ans = 0;
    for (int &i = inds[v]; i < graph[v].size(); ++i) {
      auto &e = graph[v][i];
      auto cc = min(maxF - ans, e.c - e.f);
      if (dist[e.u] <= dist[v] || !vis[e.u] || inds[e.u] ==
     graph[e.u].size() || cc < BIT)
        continue;
      auto f = dfs(e.u, cc);
      if (f != 0) {
        e.f += f, ans += f;
        graph[e.u][e.r].f -= f;
```

```
      }
      // иногда быстрее один иф, иногда другой
      if (maxF - ans < 1) break;
      // if (maxF - ans < BIT) break;
    }
    return ans;
  }

  // repeated runs are NOT independent
  void run(int s, int t) {
    S = s, T = t;
    assert(S != T);
    for (BIT = (1ll << LOG); BIT > 0; BIT >>= 1) {
      while (bfs()) {
        memset(inds.data(), 0, inds.size() * sizeof(int));
        for (auto &e : graph[S]) {
          if (inds[e.u] == graph[e.u].size() || e.c - e.f <
   BIT)
            continue;
          flow_t f = dfs(e.u, e.c - e.f);
          e.f += f, graph[e.u][e.r].f -= f;
        }
      }
    }
  }

  int addedge(int v, int u, flow_t c) { // index in graph[v]
    int i = graph[v].size();
    graph[v].pb({u, (int)graph[u].size(), 0, c});
    // если не ориентированно, то обратная capacity = c
    graph[u].pb({v, i, 0, 0});
    return i;
  }
};

void use_example() {
  Dinic dinic(n);
  for (int i = 0; i < m; ++i) {
    int v, u; flow_t c;
    cin >> v >> u >> c;
    v--, u--;
    dinic.addedge(v, u, c);
  }
  dinic.run(s, t);

  ll maxflow = 0; // may not fit flow_t
  for (auto &e : dinic.graph[s])
    maxflow += e.f;

  vector<int> cut;
  for (int i = 0; i < m; i++) {
    auto &e = edges[i];
    if (dinic.vis[e.v] && !dinic.vis[e.u])
      cut.pb(i);
  }
}
```

2.7 KCC

```
void dfs1(int v, vector<char> &used, vector<int> &topsort) {
  used[v] = 1;
  for (auto u : g[v]) {
    if (!used[u])
      dfs1(u, used, topsort);
  }
  topsort.push_back(v);
}

void dfs2(int v, int col, vector<int> &comp) {
  comp[v] = col;
  for (auto u : rg[v]) {
    if (comp[u] == -1)
      dfs2(u, col, comp);
  }
}

signed main() {
  vector<int> topsort;
  topsort.reserve(n);
  vector<char> used(n, 0);
  for (int v = 0; v < n; ++v) {
    if (!used[v])
      dfs1(v, used, topsort);
  }
  reverse(all(topsort));
  int cc = 0;
  vector<int> comp(n, -1);
  for (int i = 0; i < n; ++i) {
    if (comp[topsort[i]] == -1)
      dfs2(topsort[i], cc++, comp);
  }
}
```

2.8 Кактус

```
// кратные рёбра можно, петли нельзя
void cactus_dfs(int v, int pei, vector<vector<int>> &c2vs,
     vector<int> &e2c, vector<vector<pair<int, int>>> &g,
     vector<char> &used, vector<pair<int, int>> &st) {
  st.pb({v, pei});
  used[v] = 1;
  for (auto [u, ei] : g[v]) {
    if (ei == pei || e2c[ei] != -1)
      continue;
    if (used[u]) {
      int c = c2vs.size();
      c2vs.emplace_back();
      for (int j = st.size()-1; st[j].first != u; --j) {
        c2vs[c].pb(st[j].first);
        e2c[st[j].second] = c;
      }
      c2vs[c].pb(u);
      e2c[ei] = c;
      continue;
    }
    cactus_dfs(u, ei, c2vs, e2c, g, used, st);
  }
  st.pop_back();
}

signed main() {
  // g[v][i] = {u, ei}, граф
  // e2c[ei] = номер цикла ребра ei (либо -1)
  // c2vs[c] = список вершин цикла c (в порядке обхода)
  vector<int> e2c(m, -1);
  vector<vector<int>> c2vs;
  c2vs.reserve(m - n + 1); // 1 заменить на #комп.связ.
  {
    vector<char> used(n, 0);
    vector<pair<int, int>> st;
    st.reserve(n);
    for (int v = 0; v < n; ++v) {
      if (!used[v])
        cactus_dfs(v, -1, c2vs, e2c, g, used, st);
    }
  }
  // создание дерева по кактусу
  vector<vector<int>> ngraph(n + c2vs.size());
  for (int v = 0; v < n; ++v) {
    for (auto [u, ei] : g[v]) {
      if (e2c[ei] == -1)
        ngraph[v].pb(u);
    }
  }
  for (int c = 0; c < c2vs.size(); ++c) {
    for (int v : c2vs[c]) {
      ngraph[n + c].pb(v);
      ngraph[v].pb(n + c);
    }
  }
}
```

2.9 Минкост

```
// flow_t: capacity of one edge, total flow is ll
// cost_t: must fit n * max|w| and the answer
using flow_t = int;
using cost_t = ll;
const cost_t INFw = 1e18;
const flow_t INFf = 2e9;

struct MinCost {
  struct Edge { int v, u; flow_t f, c; cost_t w; };
  vector<Edge> edg; // edg[i ^ 1] is reverse of edg[i]
  vector<vector<int>> graph;
  vector<cost_t> dist, pot;
  vector<int> pre;
  vector<char> inQ;
  int S, T;
  MinCost(int n)
    : graph(n), dist(n), pot(n), pre(n), inQ(n) {}

  flow_t get_flow() {
    int v = T;
    if (pre[v] == -1) return 0;
    flow_t f = INFf;
    do {
      int ei = pre[v];
      Edge &e = edg[ei];
      f = min(f, e.c - e.f);
      if (f == 0) return 0;
      v = e.v;
    } while (v != S);
    v = T;
    do {
      int ei = pre[v];
      edg[ei].f += f;
      edg[ei ^ 1].f -= f;
      v = edg[ei].v;
    } while (v != S);
    return f;
  }

  void spfa() {
    fill(all(dist), INFw);
    fill(all(pre), -1);
    dist[S] = 0;
    deque<int> Q = {S};
```

```
    inQ[S] = true;
    while (!Q.empty()) {
      int v = Q.front();
      Q.pop_front();
      inQ[v] = false;
      cost_t d = dist[v];
      for (int ei : graph[v]) {
        Edge &e = edg[ei];
        if (e.f == e.c) continue;
        cost_t w = e.w + pot[v] - pot[e.u];
        if (dist[e.u] <= d + w) continue;
        pre[e.u] = ei;
        dist[e.u] = d + w;
        if (!inQ[e.u]) {
          inQ[e.u] = true;
          Q.pb(e.u);
        }
      }
    }
    for (int i = 0; i < dist.size(); ++i)
      if (dist[i] < INFw) // else pot overflows
        pot[i] += dist[i];
  }

  // returns {flow, cost}
  // repeated runs are NOT independent
  pair<ll, cost_t> run(int s, int t) {
    S = s, T = t;
    spfa(); // pot[i] = 0 // or ford_bellman
    ll flow = 0;
    while (true) {
      flow_t ff = get_flow();
      if (ff == 0) break;
      flow += ff;
      spfa(); // or dijkstra
    }
    cost_t cost = 0;
    for (int i = 0; i < edg.size(); i += 2)
      cost += edg[i].f * edg[i].w;
    return {flow, cost};
  }

  // returns id: flow on the edge = edg[id].f
  int addedge(int v, int u, flow_t c, cost_t w) {
    // reserve edg to m*2 before adding
    int i = edg.size();
    edg.pb({v, u, 0, c, w});
    edg.pb({u, v, 0, 0, -w});
    graph[v].pb(i);
    graph[u].pb(i + 1);
    return i;
  }
};

void use_example() {
  MinCost mc(n);
  for (int i = 0; i < m; ++i) {
    int v, u; flow_t c; cost_t w;
    cin >> v >> u >> c >> w;
    v--, u--;
    mc.addedge(v, u, c, w);
  }
  auto [flow, cost] = mc.run(s, t);
}
```

2.10 Мосты

```
// graph[v][i] = {u, edge_i}
void dfs(int v, int pi = -1) {
  vis[v] = 1;
  up[v] = tin[v] = timer++;
  for (auto [u, ei] : g[v]) {
    if (!vis[u]) {
      dfs(u, ei);
      up[v] = min(up[v], up[u]);
    } else if (ei != pi)
      up[v] = min(up[v], tin[u]);
    if (up[u] > tin[v]) {
      bridges.emplace_back(v, u);
      is_bridge[ei] = 1;
    }
  }
}
```

2.11 Паросочетания

```
bool dfs(int v, int c) {
  if (used[v] == c) return 0;
  used[v] = c;
  for (auto u : g[v]) {
    if (res[u] == -1) {
      res[u] = v;
      return 1;
    }
  }
  for (auto u : g[v]) {
    if (dfs(res[u], c)) {
      res[u] = v;
      return 1;
    }
  }
  return 0;
}

signed main() {
  // n - в левой доле, m - в правой
  fill(res, res + m, -1);
  fill(used, used + n, 0);
  int ans = 0;
  for (int i = 0; i < n; ++i) {
    ans += dfs(i, ans + 1);
  }
}
```

2.12 Пересечение матроидов

```
template<typename T, typename A, typename B>
vector<T> matroid_intersection(const std::vector<T> &
    ground_set, const A &matroid1, const B &matroid2) {
  // у B должны быть методы add_extra, independed_without
  // у T должно быть поле w
  // ищет минимальный по весу среди максимальных по размеру
  int n = ground_set.size();
  vector<char> in_set(n), inm1(n), inm2(n);
  vector<pair<ll, int>> dist(n);
  vector<char> inQ(n);
  vector<int> par(n), left, right;
  while (true) {
    // чтобы искать просто минимальный по весу,
    // здесь нужно прорелаксировать ответ
    A m1 = matroid1;
    B m2 = matroid2;
    left.clear();
    right.clear();
    for (int i = 0; i < n; i++)
      if (in_set[i]) {
        m1.add(ground_set[i]);
        m2.add(ground_set[i]);
        left.push_back(i);
      } else {
        right.push_back(i);
      }
    fill(all(inm1), 0);
    fill(all(inm2), 0);
    // bool found = false; // unweighted
    for (int i: right) {
      inm1[i] = m1.independed_with(ground_set[i]);
      inm2[i] = m2.independed_with(ground_set[i]);
      // if (inm1[i] && inm2[i]) { // unweighted
        // in_set[i] = 1;
        // found = true;
        // break;
      // }
    }
    // if (found) continue; // unweighted
    fill(all(dist), pair{INF, INF});
    fill(all(par), -1);
    fill(all(inQ), 0);
    deque<int> que;
    for (int i: right)
      if (inm1[i]) {
        dist[i] = {ground_set[i].w, 0};
        que.push_back(i);
        inQ[i] = 1;
      }
    while (!que.empty()) {
      // if (found) break; // unweighted
      int v = que.front();
      auto [smw, cnte] = dist[v];
      que.pop_front();
      inQ[v] = 0;
      if (in_set[v]) {
        A m = matroid1;
        for (int i: left) if (i != v) m.add(ground_set[i]);
        for (int u: right) {
          auto newdist = pair{smw + ground_set[u].w, cnte +
1};
          if (dist[u] > newdist && m.independed_with(
ground_set[u])) {
            par[u] = v;
            dist[u] = newdist;
            // if (inm2[u]) found = true; // unweighted
            if (!inQ[u]) {
              que.push_back(u);
              inQ[u] = 1;
    }}}
    } else {
      B m = m2;
      m.add_extra(ground_set[v]);
      for (auto u: left) {
        auto newdist = pair{smw - ground_set[u].w, cnte +
1};
        if (dist[u] > newdist && m.independed_without(
ground_set[u])) {
          par[u] = v;
          dist[u] = newdist;
          if (!inQ[u]) {
            que.push_back(u);
            inQ[u] = 1;
```

```
      }}}}
    }
    int bestv = -1;
    for (int v : right) {
      if (inm2[v] && dist[v].first != INF) {
        if (bestv == -1 || dist[bestv] > dist[v]) {
          bestv = v;
    }}}
    if (bestv == -1) break;
    for (; bestv != -1; bestv = par[bestv]) in_set[bestv] ^=
     1;
  }
  vector<T> res;
  for (int i = 0; i < n; i++) if (in_set[i]) res.push_back(
     ground_set[i]);
  return res;
}
```

2.13 Простой путь с заходом

```
// Кратчайший простой путь s->t через все need, неор граф.
// s==t => непустой цикл (даже при |need|=0).
// O(2^|need| (n+m) ans)
ull path_mul(ull a, ull b) {
  ull r = 0;
  for (; b; b >>= 1, a = (a << 1) ^ (a >> 63) * 27)
    r ^= a * (b & 1);
  return r;
}

// Возвращает {число рёбер, первая вершина после s}, либо
    {-1,-1} если нет.
pair<int, int> find_path_through_specified(int n,
    vector<pair<int, int>> edges, int s, int t,
    vector<int> need) {
  vector<int> bit(n);
  int k = 0;
  for (auto v : need) {
    if (v == s || v == t || bit[v]) continue;
    bit[v] = 1 << k++;
  }
  int masks = 1 << k;
  struct Edge { int v, u; ull w; };
  vector<Edge> es;
  vector<int> from, to;
  for (auto [v, u] : edges) {
    if ((v == s && u == t) || (v == t && u == s)) {
      if (k == 0) return {1, t};
      continue;
    }
    if (v == s || u == s) from.push_back(v ^ u ^ s);
    if (v == t || u == t) to.push_back(v ^ u ^ t);
    if (v != s && v != t && u != s && u != t) {
      ull w = rnd();
      es.push_back({v, u, w});
      es.push_back({u, v, w});
    }
  }
  int m = es.size();
  vector<vector<ull>> dp(masks, vector<ull>(n));
  vector<vector<ull>> dp_e(masks, vector<ull>(m));
  auto ndp = dp, ndp_e = dp_e;
  for (auto v : to) dp[bit[v]][v] = rnd();
  for (int len = 1; len < n; ++len) {
    if (s != t || len >= 2) {
      for (auto v : from) {
        if (dp[masks - 1][v]) return {len + 1, v};
      }
    }
    if (len + 1 == n) break;
    for (auto &r : ndp) fill(all(r), 0);
    for (auto &r : ndp_e) fill(all(r), 0);
    for (int mask = 0; mask < masks; ++mask) {
      for (int id = 0; id < m; ++id) {
        auto [v, u, w] = es[id];
        if (bit[u] && (mask & bit[u])) continue;
        int nmask = mask | bit[u];
        ull x = dp[mask][v] ^ dp_e[mask][id ^ 1];
        if (!x) continue;
        x = path_mul(x, w);
        ndp[nmask][u] ^= x;
        ndp_e[nmask][id] ^= x;
      }
    }
    swap(dp, ndp);
    swap(dp_e, ndp_e);
  }
  return {-1, -1};
}
```

2.14 Точки сочленения

```
void dfs(int v, int p = -1) {
  vis[v] = 1;
  up[v] = tin[v] = timer++;
  bool is_artic = false;
  int child = 0;
  for (auto u : g[v]) {
    if (!vis[u]) {
      child++;
      dfs(u, v);
      up[v] = min(up[v], up[u]);
      is_artic |= p != -1 && up[u] >= tin[v];
    } else
      up[v] = min(up[v], tin[u]);
  }
  is_artic |= p == -1 && child >= 2;
  if (is_artic)
    articulation_points.insert(v);
}
```

2.15 Эйлеров цикл

```
// Эйлеров путь/цикл в компоненте связности s. Возвращает инде
    ксы рёбер. Если пути/цикла нет, то алгос найдёт фигню.
// Если неориентированный граф, то edges[ei] и edges[ei ^ 1] -
     обратные друг к другу рёбра.
// edges[graph[v][i]] = {v, u}
vector<int> eulerpath1(int s, vector<vector<int>> &graph,
    vector<pair<int, int>> &edges, vector<char> &used, vector
    <int> &start) {
  vector<pair<int, int>> st = {{-1, s}};
  vector<int> res;
  while (!st.empty()) {
    auto [ei, v] = st.back();
    while (start[v] < graph[v].size() && used[graph[v][start[v
    ]]])
      start[v]++;
    if (start[v] == graph[v].size()) {
      if (ei != -1) res.push_back(ei);
      st.pop_back();
    } else {
      int ej = graph[v][start[v]++];
      used[ej] = true;
      used[ej ^ 1] = true; // Удалить если ориент. граф
      st.emplace_back(ej, edges[ej].second);
    }
  }
  reverse(all(res));
  return res;
}

vector<char> used(edges.size(), false);
vector<int> start(graph.size(), 0);
for (int v = 0; v < graph.size(); ++v) {
  // Если ориентированный граф, второе условие заменить на
     cnt_in[v] >= cnt_out[v]
  if (start[v]==graph[v].size() || graph[v].size()%2==0)
    continue;
  auto path = eulerpath1(v, graph, edges, used, start);
}
for (int v = 0; v < graph.size(); ++v) {
  if (start[v] == graph[v].size())
    continue;
  auto cycle = eulerpath1(v, graph, edges, used, start);
}
```

3 ДП

3.1 CHT

```
struct line {
  ll k, b;
  i128 eval(ll x) { return (i128)k * x + b; }
};

// check if b is redudant, exact
bool redundant(line a, line b, line c) {
  return (a.b - b.b) * (i128)(c.k - b.k)
      >= (b.b - c.b) * (i128)(b.k - a.k);
}
// floating point alternative, if k and b are not integer
dbl inter(line a, line b) { return (a.b-b.b)/(dbl)(b.k-a.k); }
bool redundant(line a, line b, line c) { return inter(a,b)>=
     inter(b,c); }

// |k|, |b| <= 1e18; query answer must fit long long
struct ConvexHull {
  // for min: k decreasing (non-increasing)
  // for max: k increasing (non-decreasing)
  vector<line> st;

  void add(line a) {
    if (!st.empty() && st.back().k == a.k) {
      if (st.back().b < a.b) st.pop_back(); // for max
    //if (st.back().b > a.b) st.pop_back(); // for min
      else return;
    }
    while (st.size() > 1 && redundant(st[st.size()-2], st.back
     (), a))
      st.pop_back();
    st.pb(a);
  }

  ll get_val(ll x) {
```

```
    if (st.empty()) return -INF; // +INF for min
    int l = -1, r = ssize(st) - 1;
    while (r - l > 1) {
      int m = (l + r) / 2;
      if (st[m].eval(x) < st[m + 1].eval(x)) l = m; // > for
     min
      else r = m;
    }
    return st[r].eval(x);
  }
};
```

3.2 Li Chao

```
struct Line {
  ll k, b;
  ll operator()(ll x) const { return k * x + b; }
};

// LiChao for maximum at x
struct LiChao {
  struct Node {
    int l, r;
    Line ln;
  };
  ll L, R;
  int root;
  vector<Node> t;

  LiChao(ll L, ll R, int maxsz=0) : L(L), R(R), root(0) {
    t.reserve(maxsz + 1);
    t.pb({0, 0, {0, -INF}}); // t[0] = empty. +INF for min
  }

  int add_on_seg(int v, ll l, ll r, ll ql, ll qr, Line ln) {
    if (qr <= l || r <= ql) { return v; }
    if (ql <= l && r <= qr) { return add(v, l, r, ln); }
    if (!v) { v = t.size(), t.pb({0, 0, {0, -INF}}); }
    ll m = (l + r) >> 1; // NOT (l+r)/2
    t[v].l = add_on_seg(t[v].l, l, m, ql, qr, ln);
    t[v].r = add_on_seg(t[v].r, m, r, ql, qr, ln);
    return v;
  }
  void add_on_seg(ll ql, ll qr, Line ln) { // [ql,qr)
    root = add_on_seg(root, L, R, ql, qr, ln);
  }

  int add(int v, ll l, ll r, Line ln) {
    if (!v) {
      v = t.size();
      t.pb({0, 0, ln});
      return v;
    }
    ll m = (l + r) >> 1; // NOT (l+r)/2
    if (ln(m) > t[v].ln(m)) swap(ln, t[v].ln); // "<" for min
    if (r - l == 1 || ln.k == t[v].ln.k) return v;
    if (ln.k < t[v].ln.k) t[v].l = add(t[v].l, l, m, ln); //
     ">" for min
    else t[v].r = add(t[v].r, m, r, ln);
    return v;
  }
  void add(Line nw) { root = add(root, L, R, nw); }

  ll get(ll x) {
    ll ans = -INF; // +INF for min
    int v = root;
    ll l = L, r = R;
    while (v) {
      ans = max(ans, t[v].ln(x)); // "min" for min
      ll m = (l + r) >> 1; // NOT (l+r)/2
      if (x < m) v = t[v].l, r = m;
      else v = t[v].r, l = m;
    }
    return ans;
  }
};
```

3.3 SOS-dp

```
// dp initial fill, a[] is given array, mb extra zeros
for (int i = 0; i < (1 << N); i++) {
  dp[i] = a[i];
}

// Classic SOS-dp, goal: dp[mask] = \sum a[submasks of mask]
for (int i = 0; i < N; i++) {
  for (int mask = 0; mask < (1 << N); mask++) {
    if ((mask >> i) & 1) {
      dp[mask] += dp[mask ^ (1 << i)];
    }
  }
}

// Overmasks SOS-dp, goal: dp[mask] = \sum a[overmasks of mask
    ]
for (int i = 0; i < N; i++) {
  for (int mask = (1 << N) - 1; mask >= 0; mask--) {
    if (((mask >> i) & 1) == 0) {
      dp[mask] += dp[mask ^ (1 << i)];
    }
  }
}

// to inverse SOS-dp (restore original array by SOS-dp array):
// use same code, but -= instead of += in dp transitions
```

3.4 UnlimitedKnapsack

```
void unlimited_knapsack(int n, int T, vector<int> &v) {
  // O(n^2 log n + T)
  // v[1..n], v[i] --- best value of a single item with weight
     =i (-inf for non-existent)
  vector<int> items;
  for (int i = 1; i <= n; i++)
    if (v[i] > -inf) items.pb(i);
  sort(all(items), [&](int i, int j) {
    return v[i] * j > v[j] * i;
  });
  v.resize(T + 1, -inf);
  v[0] = 0;
  for (int i = 2; i <= T; i++) {
    int tick = 0;
    for (auto j: items) {
      if (i > j) {
        v[i] = max(v[i], v[j] + v[i - j]);
      }
      if (tick * i > 3 * n * n) break; // apparently 2 works
     as well, but proved bound is 3
      tick++;
    }
  }
  // v[i] --- best total value of items with total weight=i
  // weight i is unreachable if `v[i] < -inf/2` (not `== -inf
     `)
}
```

3.5 НВП

```
// strict LIS, returns indices
vector<int> calc_lis(const vector<int> &a) {
  vector<int> dp, ind, par(a.size());
  dp.reserve(a.size()), ind.reserve(a.size());
  for (int i = 0; i < a.size(); ++i) {
    // upper_bound for non-strict LIS
    int len = lower_bound(all(dp), a[i]) - dp.begin();
    par[i] = len ? ind[len - 1] : -1;
    if (len == dp.size()) {
      dp.pb(a[i]);
      ind.pb(i);
    } else {
      dp[len] = a[i];
      ind[len] = i;
    }
  }
  int len = dp.size();
  vector<int> ans(len);
  if (len==0) return ans;
  for (int i = len-1, j=ind.back(); i>=0; j=par[j], --i)
    ans[i] = j;
  return ans;
}
```

4 Деревья

4.1 Centroid

```
// g: tree or forest
// par[v] = parent in centroid trees; -1 for roots
// level[root] = 1
struct Centroid {
  const vector<vector<int>> &g;
  vector<int> levels, _szs, par;
  Centroid(const vector<vector<int>> &g) : g(g),
      levels(g.size()), _szs(g.size()), par(g.size()) {
    for (int v = 0; v < g.size(); ++v)
      if (levels[v] == 0) build(v);
  }

  int _calcsizes(int v, int p) {
    int sz = 1;
    for (int u : g[v]) {
      if (u != p && levels[u] == 0)
        sz += _calcsizes(u, v);
    }
    return _szs[v] = sz;
  }

  void build(int v, int lvl = 1, int p = -1) {
    int sz = _calcsizes(v, -1);
    int nxt = v, prv;
    while (nxt != -1) {
      prv = v, v = nxt, nxt = -1;
      for (int u : g[v]) {
        if (u != prv && levels[u] == 0 && _szs[u] * 2 >= sz)
```

```
        nxt = u;
      }
    }
    levels[v] = lvl;
    par[v] = p;
    for (int u : g[v]) {
      if (levels[u] == 0)
        build(u, lvl + 1, v);
    }
    // calc smth for centroid v: dfs by levels[u]>=lvl
    // _szs[] are NOT subtree sizes (it's trash)
  }
};
```

4.2 HLD

```
int par[MAXN], sizes[MAXN];
int pathup[MAXN];
int tin[MAXN], tout[MAXN];
int timer;

int dfs1_hld(int v, int p) {
  par[v] = p;
  int sz = 1;
  for (int i = 0; i < graph[v].size(); ++i) {
    int u = graph[v][i];
    if (u == p) {
      swap(graph[v][i--], graph[v].back());
      graph[v].pop_back();
      continue;
    }
    sz += dfs1_hld(u, v);
  }
  return sizes[v] = sz;
}

void dfs2_hld(int v, int up) {
  tin[v] = timer++;
  pathup[v] = up;
  for (int i = 1; i < graph[v].size(); ++i) {
    if (sizes[graph[v][i]] > sizes[graph[v][0]])
      swap(graph[v][i], graph[v][0]);
  }
  for (int u : graph[v])
    dfs2_hld(u, u == graph[v][0] ? up : u);
  tout[v] = timer;
}

bool is_ancestor(int v, int p) {
  return tin[p] <= tin[v] && tout[v] <= tout[p];
}

// get_hld полностью аналогичный
void update_hld(int v, int u, int ARG) {
  for (int _ = 0; _ < 2; ++_) {
    while (!is_ancestor(u, pathup[v])) {
      int vup = pathup[v];
      ST.update(0, 0, timer, tin[vup], tin[v] + 1, ARG);
      v = par[vup];
    }
    swap(v, u);
  }
  if (tin[v] > tin[u])
    swap(v, u);
  // v = lca
  ST.update(0, 0, timer, tin[v], tin[u] + 1, ARG);
}

signed main() {
  dfs1_hld(0, -1);
  dfs2_hld(0, 0);
  ST.build();
  // your code here
}
```

4.3 Link-cut

```
struct Node {
  Node *ch[2];
  Node *p;
  bool rev;
  int sz;

  Node() {
    ch[0] = nullptr;
    ch[1] = nullptr;
    p = nullptr;
    rev = false;
    sz = 1;
  }
};

int size(Node *v) {
  return (v ? v->sz : 0);
}

int chnum(Node *v) {
  return v->p->ch[1] == v;
}

bool isroot(Node *v) {
  return v->p == nullptr || v->p->ch[chnum(v)] != v;
}

void push(Node *v) {
  if (v->rev) {
    if (v->ch[0])
      v->ch[0]->rev ^= 1;
    if (v->ch[1])
      v->ch[1]->rev ^= 1;
    swap(v->ch[0], v->ch[1]);
    v->rev = false;
  }
}

void pull(Node *v) {
  v->sz = size(v->ch[1]) + size(v->ch[0]) + 1;
}

void attach(Node *v, Node *p, int num) {
  if (p)
    p->ch[num] = v;
  if (v)
    v->p = p;
}

void rotate(Node *v) {
  Node *p = v->p;
  push(p);
  push(v);
  int num = chnum(v);
  Node *u = v->ch[1 - num];
  if (!isroot(v->p))
    attach(v, p->p, chnum(p));
  else
    v->p = p->p;
  attach(u, p, num);
  attach(p, v, 1 - num);
  pull(p);
  pull(v);
}

void splay(Node *v) {
  push(v);
  while (!isroot(v)) {
    if (!isroot(v->p)) {
      if (chnum(v) == chnum(v->p))
        rotate(v->p);
      else
        rotate(v);
    }
    rotate(v);
  }
}

void expose(Node *v) {
  splay(v);
  v->ch[1] = nullptr;
  pull(v);
  while (v->p != nullptr) {
    Node *p = v->p;
    splay(p);
    attach(v, p, 1);
    pull(p);
    splay(v);
  }
}

void makeroot(Node *v) {
  expose(v);
  v->rev ^= 1;
  push(v);
}

void link(Node *v, Node *u) {
  makeroot(v);
  makeroot(u);
  u->p = v;
}

void cut(Node *v, Node *u) {
  makeroot(u);
  makeroot(v);
  v->ch[1] = nullptr;
  u->p = nullptr;
  pull(v);
}

int get(Node *v, Node *u) {
  makeroot(u);
  makeroot(v);
  Node *w = u;
  while (!isroot(w))
    w = w->p;
  return (w == v ? size(v) - 1 : -1);
}

const int MAXN = 100010;
Node nds[MAXN];
```

```
int main() {
  int n, q;
  cin >> n >> q;
  while (q--) {
    string s;
    int a, b;
    cin >> s >> a >> b;
    a--, b--;
    if (s[0] == 'g')
      cout << get(nds + a, nds + b) << '\n';
    else if (s[0] == 'l')
      link(nds + a, nds + b);
    else
      cut(nds + a, nds + b);
  }
}
```

5 Другое

5.1 Fast hashmap

```
struct Hashmap {
  using K = int; using V = int; using cK = const K;
  static constexpr K EMP = INT_MIN; // forbidden key
  static constexpr unsigned RND = 1791791791; // random odd
  int lg, m;
  vector<K> kk; vector<V> vv; // K kk[SZ]; V vv[SZ];
  K *k; V *v;

  Hashmap(int n) { // max size
    lg = 32 - __builtin_clz(n) + 2; // <8n, mem-perf tradeoff
    m = (1<<lg)-1;
    kk.assign(m+1, EMP); vv.resize(m+1); // fill(kk,kk+s,EMP);
    k=kk.data(), v=vv.data(); // k=kk,v=vv;
  }
  // must have if copying hashmap (because of pointers k,v):
  Hashmap(const Hashmap &o) { *this=o; }
  Hashmap &operator=(const Hashmap &o) {
    lg=o.lg, m=o.m; kk=o.kk, vv=o.vv;
    k=kk.data(), v=vv.data(); return *this;
  }
  int h(cK &x) const { return unsigned(x)*RND>>(32-lg); }
  int pos(cK &x) const {
    int i = h(x);
    while (k[i]!=EMP && k[i]!=x) i=(i+1)&m;
    return i;
  }
  V* find(cK &x) const { int i=pos(x); return k[i]==x?v+i:0; }
  bool contains(cK &x) const { return find(x); }
  const V &operator[](cK &x) const { return *find(x); }
  V &operator[](cK &x) {
    int i = pos(x);
    // Use =V() if need default constructor and erase
    //   (e.g. `cnt.erase(x); cnt[y]++;`)
    if (k[i]!=x) k[i]=x; //, v[i]=V();
    return v[i];
  }
  bool erase(cK &x) {
    int i = pos(x);
    if (k[i]!=x) return 0;
    for (int j=(i+1)&m; k[j]!=EMP; j=(j+1)&m) {
      int r = h(k[j]);
      if (((j-r)&m) > ((i-r)&m))
        k[i]=move(k[j]), v[i]=move(v[j]), i=j;
    }
    return k[i]=EMP, 1;
  }
};
```

5.2 Fast mod

```
// Быстрое взятие по НЕ константному модулю (в 2-4 раза быстре
    е)
struct FastMod {
  ull b, m;
  FastMod(ull b) : b(b), m(-1ULL / b) {}
  ull mod(ull a) const {
      ull r = a - (ull)((__uint128_t(m) * a) >> 64) * b;
      return r;  // r in [0, 2b) // ≈ x3.5 speed
      return r >= b ? r - b : r; // ≈ x3 speed
  }
}; // Usage:
// FastMod F(m);
// ull x_mod_m = F.mod(x);
```

5.3 Fast sort

```
// n=1e6, a<1e18, B=2^12, iters=5, time=17ms
// n=1e7, a<1e18, B=2^12, iters=5, time=190ms
// n=1e8, a<1e18, B=2^12, iters=5, time=1.9s
// n=1e6, a<1e9, B=2^10, iters=3, time=10ms
// n=1e7, a<1e9, B=2^10, iters=3, time=120ms
// n=1e8, a<1e9, B=2^10, iters=3, time=1.2s
void radix_sort(vector<ll> &a) {
  // оптимальный B ≈ 2^10 - 2^12, не обязательно степень 2
  // 0 ≤ a < B^iters
  // далее можно уменьшить B сохраняя такой же iters
  const signed B = 1 << 12;
  static signed c[B + 1];
  vector<ll> b(a.size());
#define PASS(key) { \
  memset(c, 0, sizeof c); \
  for (auto x : a) ++c[(key)+1]; \
  for (signed i = 1; i <= B; ++i) { c[i]+=c[i-1]; } \
  for (auto x : a) b[c[key]++] = x; \
  swap(a,b); \
}
  PASS(x%B);
  PASS(x/B%B);
  PASS(x/B/B%B);
  PASS(x/B/B/B%B);
  PASS(x/B/B/B/B);
#undef PASS
}
```

5.4 attribute_packed

```
struct Kek {
  int a;
  char b;
  // char[3]
  int c;
} __attribute__((packed));
// sizeof = 9 (instead of 12)
```

5.5 custom_bitset

```
// __builtin_ctz = Count Trailing Zeroes
// __builtin_clz = Count Leading Zeroes
// both are UB in gcc when pass 0
struct custom_bitset {
  vector<uint64_t> bits;
  int b, n;

  custom_bitset(int bb = 0) { init(bb); }
  void init(int bb) {
    b = bb, n = (b + 63) / 64;
    bits.assign(n, 0);
  }
  void clear() { b = n = 0, bits.clear(); }
  void reset() { bits.assign(n, 0); }
  void _clean() {
    // Reset all bits after `b`.
    if (b % 64)
      bits.back() &= (1LLU << (b % 64)) - 1;
  }
  bool get(int i) const {
    return bits[i / 64] >> (i % 64) & 1;
  }
  void set(int i, bool value) {
    // assert(0 <= i && i < b);
    bits[i / 64] &= ~(1LLU << (i % 64));
    bits[i / 64] |= uint64_t(value) << (i % 64);
  }

  // Simulates `bs1 |= bs2 << s;`
  // `|=` can be replaced with `^=`, `&=`, `=`
  void or_shift_left(int s){return or_shift_left(*this,s);}
  void or_shift_left(const custom_bitset &o, int s) {
    // assert(s >= 0);
    int d = s/64, r = s%64;
    if (r == 0) {
      for (int i = n-1; i >= d; --i)
        bits[i] |= o.bits[i-d];
    } else {
      for (int i = n-1; i >= d+1; --i)
        bits[i] |= o.bits[i-d-1]>>(64-r) | o.bits[i-d]<<r;
      if (d < n)
        bits[d] |= o.bits[0]<<r;
    }
    // if `&=`, `=`
    //fill(bits.begin(), bits.begin() + min(d, n), 0);
    _clean();
  }

  // Simulates `bs1 |= bs2 >> s;`
  // `|=` can be replaced with `^=`, `&=`, `=`
  void or_shift_right(int s){return or_shift_right(*this,s);}
  void or_shift_right(const custom_bitset &o, int s) {
    // assert(s >= 0);
    int d = s/64, r = s%64;
    if (r == 0) {
      for (int i = d; i < n; ++i)
        bits[i-d] |= o.bits[i];
    } else {
      for (int i = 0; i < n-d-1; ++i)
        bits[i] |= o.bits[i+d+1]<<(64-r) | o.bits[i+d]>>r;
      if (d < n)
        bits[n-d-1] |= o.bits[n-1]>>r;
    }
    // if `&=`, `=`
    //fill(bits.end() - min(d, n), bits.end(), 0);
```

```
    _clean();
  }

  // find min j, that j >= i and bs[j] = 1;
  int find_next(int i) {
    i = max(i,0);
    if (i >= b) return b;
    int d = i/64, r = i%64;
    auto x = bits[d]>>r;
    if (x != 0)
      return i + __builtin_ctzll(x);
    for (auto k = d+1; k < n; ++k) {
      if (bits[k] != 0)
        return 64*k + __builtin_ctzll(bits[k]);
    }
    return b;
  }

  // `|=` can be replaced with `&=`, `^=`
  custom_bitset &operator|=(const custom_bitset &other){
    // assert(b == other.b);
    for (int i = 0; i < n; i++)
      bits[i] |= other.bits[i];
    return *this;
  }
};
```

5.6 Аллокатор Копелиовича

```
// insert code BEFORE includes
#include <cassert>
using namespace std;
const size_t MAX_MEM = 1e8; // ~100mb
// set ALIGN=1 if care about ML, but not UB
const size_t ALIGN = __STDCPP_DEFAULT_NEW_ALIGNMENT__;
alignas(ALIGN) unsigned char mem[MAX_MEM];
size_t mpos = 0;

void *operator new(size_t n) {
  n = ((n ? n : 1) + ALIGN - 1) & ~(ALIGN - 1);
  mpos += n;
//  assert(mpos <= MAX_MEM);
  return (void *)(mem + mpos - n);
}
void operator delete(void *) noexcept {}
void operator delete(void *, size_t) noexcept {}
```

5.7 Альфа-бета отсечение

```
int alphabeta(int player, int alpha, int beta, int depth) {
  if (depth == 0) {
    // return current position score
  }
  if (player == 0) { // maximization player
    int val = -INF;
    for (auto move : possible_moves) {
      val = max(val, alphabeta(1, alpha, beta, depth - 1));
      if (val >= beta) break;
      alpha = max(alpha, val);
    }
    return val;
  } else {
    int val = INF;
    for (auto move : possible_moves) {
      val = min(val, alphabeta(0, alpha, beta, depth - 1));
      if (val <= alpha) break;
      beta = min(beta, val);
    }
    return val;
  }
}
```

5.8 Отжиг

```
const double lambda = 0.999;
double temperature = 1;
mt19937 rnd(777);

double gen_rand_01() {
  return rnd() / (double) mt19937::max();
}
bool f(int delta) {
  return exp(-delta / temperature) > gen_rand_01();
}
void make_change() {
  temperature *= lambda;
  // calc change score
  if (change_score <= 0 || f(change_score)) {
    score += change_score;
    // make change
  }
}
```

6 Математика

6.1 AdivB cmp CdivD

```
// a/b vs c/d: -1 = less, 0 = equal, 1 = greater
// O(1) cross-multiplication
char compare(ll a, ll b, ll c, ll d) {
  i128 l = (i128)a * d, r = (i128)c * b;
  return ((l>r) - (l<r)) * sign(b) * sign(d);
}
// O(log), only a,b,c,d must fit into type
char compare(i128 a, i128 b, i128 c, i128 d) {
  if (a/b != c/d) return a/b < c/d ? -1 : 1;
  a %= b, c %= d;
  if (a == 0) return -sign(c) * sign(d);
  if (c == 0) return sign(a) * sign(b);
  return compare(d,c,b,a)*sign(a)*sign(b)*sign(c)*sign(d);
}
```

6.2 Berlecamp

```
int getkfps(vector<ll> p, vector<ll> q, ll k) {
  // assert(q[0] != 0);
  while (k) {
    auto f = q;
    for (int i = 1; i < (int) f.size(); i += 2) {
      f[i] = (MOD - f[i] % MOD) % MOD;
    }
    auto p2 = convMod(p, f);
    auto q2 = convMod(q, f);
    p.clear(), q.clear();
    for (int i = k % 2; i < (int) p2.size(); i += 2) {
      p.pb(p2[i]);
    }
    for (int i = 0; i < (int) q2.size(); i += 2) {
      q.pb(q2[i]);
    }
    k >>= 1;
  }
  return p.empty() ? 0 : (p[0] * inverse(q[0])) % MOD;
}

// a - initials values of sequence, s - result of berlekamp on
    a
int kth_term(vector<ll> &a, vector<ll> s, ll k) {
  int d = ssize(s) - 1;
  s[0] = MOD - 1;
  while (s.back() == 0) {
    s.pop_back();
  }
  for (auto &el: s) {
    el = (MOD - el % MOD) % MOD;
  }
  vector<ll> p(d);
  copy(a.begin(), a.begin() + d, p.begin());
  p = convMod(p, s);
  p.resize(d);
  return getkfps(p, s, k);
}

vector<ll> berlekamp_massey(vector<ll> a) {
  // given a[0]...a[n], returns sequence s[1]..s[k] s.t a[i] =
      a[i-1] \cdot s[1] + \ldots + a[i-k] \cdot s[k]
  vector <ll> ls, s;
  int lf = 0, d = 0;
  for (int i = 0; i < a.size(); ++i) {
    ll t = 0;
    for (int j = 0; j < s.size(); ++j) {
      t = (t + 1ll * a[i - j - 1] * s[j]) % MOD;
    }
    if ((t - a[i]) % MOD == 0)continue;
    if (s.empty()) {
      s.resize(i + 1);
      lf = i;
      d = (t - a[i]) % MOD;
      continue;
    }
    ll k = -(a[i] - t) * inverse(d) % MOD;
    vector<ll> c(i - lf - 1);
    c.push_back(k);
    for (auto &j: ls)
      c.push_back(-j * k % MOD);
    if (c.size() < s.size())
      c.resize(s.size());
    for (int j = 0; j < s.size(); ++j) {
      c[j] = (c[j] + s[j]) % MOD;
    }
    if (i - lf + (int) ls.size() >= (int) s.size()) {
      tie(ls, lf, d) = make_tuple(s, i, (t - a[i]) % MOD);
    }
    s = c;
  }
  s.insert(s.begin(), 0); // fictive s[0] = 0
  for (auto &i: s)
    i = (i % MOD + MOD) % MOD;
  return s;
}
```

6.3 FFT

```
const double PI = acos(-1);
const int LOG = 20;
const int MAXN = 1 << LOG;

//using comp = complex<double>;
struct comp {
  double x, y;
  comp() : x(0), y(0) {}
  comp(double x, double y) : x(x), y(y) {}
  comp(int x) : x(x), y(0) {}
  comp operator+(const comp &o) const { return {x + o.x, y + o
     .y}; }
  comp operator-(const comp &o) const { return {x - o.x, y - o
     .y}; }
  comp operator*(const comp &o) const { return {x * o.x - y *
     o.y, x * o.y + y * o.x}; }
  comp operator/(const int k) const { return {x / k, y / k}; }
  comp conj() const { return {x, -y}; }
};

comp OMEGA[MAXN + 10];
int tail[MAXN + 10];
bool fftinit = false;

comp omega(int n, int k) {
  return OMEGA[MAXN / n * k];
}

int gettail(int x, int lg) {
  return tail[x] >> (LOG - lg);
}

void calcomega() {
  for (int i = 0; i < MAXN; ++i) {
    double x = 2 * PI * i / MAXN;
    OMEGA[i] = {cos(x), sin(x)};
  }
}

void calctail() {
  tail[0] = 0;
  for (int i = 1; i < MAXN; ++i)
    tail[i] = (tail[i/2]/2) | ((i & 1) << (LOG - 1));
}

void fft(vector<comp> &A, int lg) {
  if (!fftinit) calcomega(), calctail(), fftinit = 1;
  int n = A.size();
  for (int i = 0; i < n; ++i) {
    int j = gettail(i, lg);
    if (i < j)
      swap(A[i], A[j]);
  }
  for (int len = 2; len <= n; len *= 2) {
    for (int i = 0; i < n; i += len) {
      for (int j = 0; j < len / 2; ++j) {
        auto v = A[i + j];
        auto u = A[i + j + len / 2] * omega(len, j);
        A[i + j] = v + u;
        A[i + j + len / 2] = v - u;
      }
    }
  }
}

// optional, can be skipped
void fft2(vector<comp> &A, vector<comp> &B, int lg) {
  int n = A.size();
  vector<comp> C(n);
  for (int i = 0; i < n; ++i) {
    C[i].x = A[i].x;
    C[i].y = B[i].x;
  }
  fft(C, lg);
  C.push_back(C[0]);
  for (int i = 0; i < n; ++i) {
    A[i] = (C[i] + C[n - i].conj()) / 2;
    B[i] = (C[i] - C[n - i].conj()) / 2 * comp(0, -1);
  }
}

void invfft(vector<comp> &A, int lg) {
  int n = 1 << lg;
  fft(A, lg);
  for (auto &el : A)
    el = el / n;
  reverse(A.begin() + 1, A.end());
}

vector<int> mul(vector<int> &a, vector<int> &b) {
  if (a.empty() || b.empty())
    return {};
  int lg = 32-__builtin_clz(max<int>(1,a.size()+b.size()-2));
  int n = 1 << lg;
  vector<comp> A(n, 0), B(n, 0);
  for (int i = 0; i < a.size(); ++i)
    A[i] = a[i];
  for (int i = 0; i < b.size(); ++i)
    B[i] = b[i];
//  fft2(A, B, lg);
  fft(A, lg);
  fft(B, lg);
  for (int i = 0; i < n; ++i)
    A[i] = A[i] * B[i];
  invfft(A, lg);
  vector<int> c(n);
  for (int i = 0; i < n; ++i)
    c[i] = round(A[i].x);
  while (!c.empty() && c.back() == 0)
    c.pop_back();
  return c;
}
```

6.4 Floor Sum, Min Rem

```
// f(x) = (a*x+b) mod m in [0,m); ll = __int128.
// S = {0} U {x > 0 : f(x) < min_{0 <= j < x} f(j)}.
// {X,DX}: S = {0} U {X[i] + t*DX[i] : 0 <= i < DX.size(),
//   1 <= t <= (X[i+1]-X[i])/DX[i]}; X[0] = 0; O(log m).
pair<vector<ll>, vector<ll>> min_of_linear_segments(ll a, ll b
    , ll m) {
  a = (a % m + m) % m, b = (b % m + m) % m;
  ll g = __gcd(a, m);
  ll A = a / g, M = m / g, y = b / g;
  ll q = 1, s = 1, dl = M - A, dr = A, x = 0;
  vector<ll> X = {0}, DX;
  while (y) {
    ll k = dr / dl;
    dr %= dl;
    if (!dr) --k, dr = dl;
    s += k * q;
    while (true) {
      k = max<ll>(0, (dl - y + dr - 1) / dr);
      if (dl - k * dr <= 0) break;
      dl -= k * dr;
      q += k * s;
      k = y / dl;
      y -= k * dl, x += q * k;
      X.pb(x), DX.pb(q);
    }
    k = dl / dr;
    dl -= k * dr;
    q += k * s;
  }
  return {X, DX};
}
pair<vector<ll>, vector<ll>> max_of_linear_segments(ll a, ll b
    , ll m) {
  return min_of_linear_segments(-a, m - 1 - b, m);
}
// min {x >= 0 : l <= f(x) <= r}; -1, если множество пусто.
// 0 <= l <= r < m; O(log m).
ll min_rem(ll a, ll b, ll m, ll l, ll r) {
  if (l > r) return -1;
  ll len = r - l + 1;
  b = ((b - l) % m + m) % m;
  if (b < len) return 0;
  auto [X, DX] = min_of_linear_segments(a, b, m);
  for (ll i = 0; i < DX.size(); ++i) {
    ll y2 = ((a * X[i + 1] + b) % m + m) % m;
    if (y2 >= len) continue;
    ll y1 = ((a * X[i] + b) % m + m) % m;
    ll d = (y1 - y2) / ((X[i + 1] - X[i]) / DX[i]);
    return X[i] + ((y1 - len) / d + 1) * DX[i];
  }
  return -1;
}
// {x,y}: y = min_{0 <= i < n} f(i), x = min i: f(i) = y.
// n > 0; O(log m).
pair<ll, ll> min_mod(ll n, ll a, ll b, ll m) {
  a = (a % m + m) % m, b = (b % m + m) % m;
  auto [X, DX] = min_of_linear_segments(a, b, m);
  ll x = 0;
  for (ll i = 0; i < DX.size() && X[i] < n; ++i)
    x = X[i] + (min(X[i + 1], n - 1) - X[i]) / DX[i] * DX[i];
  return {x, (a * x + b) % m};
}
// {x,y}: y = max_{0 <= i < n} f(i), x = min i: f(i) = y.
// n > 0; O(log m).
pair<ll, ll> max_mod(ll n, ll a, ll b, ll m) {
  auto [x, y] = min_mod(n, -a, m - 1 - b, m);
  return {x, m - 1 - y};
}
// {x,y}: y = min_{L <= i < R} f(i), x = min i: f(i) = y.
// L < R; O(log m).
pair<ll, ll> min_of_linear(ll L, ll R, ll a, ll b, ll m) {
  auto [x, y] = min_mod(min(R - L, m), a, a * L + b, m);
  return {L + x, y};
}
// sum_{0 <= x < n} floor((a*x+b)/m); O(log m).
ll floor_sum(ll n, ll a, ll b, ll m) {
  ll ans = 0;
  while (n) {
    ll q = a / m - (a % m < 0);
    a -= q * m;
    ans += n * (n - 1) / 2 * q;
    q = b / m - (b % m < 0);
    b -= q * m;
    ans += n * q;
    ll y = a * n + b;
    if (y < m) break;
```

```
    n = y / m, b = y % m;
    swap(a, m);
  }
  return ans;
}
// sum_{L <= x < R} floor((a*x+b)/m); O(log m).
ll floor_sum_of_linear(ll L, ll R, ll a, ll b, ll m) {
  return floor_sum(R - L, a, a * L + b, m);
}
// |{x : 0 <= x < n, l <= f(x) <= r}|.
// 0 <= l <= r < m; l > r -> 0; O(log m).
ll count_mod(ll n, ll a, ll b, ll m, ll l, ll r) {
  if (l > r) return 0;
  a = (a % m + m) % m, b = (b % m + m) % m;
  return floor_sum(n, a, b - l, m) -
         floor_sum(n, a, b - r - 1, m);
}
// sum_{0 <= x < n} f(x); O(log m).
ll sum_mod(ll n, ll a, ll b, ll m) {
  a = (a % m + m) % m, b = (b % m + m) % m;
  return n * (n - 1) / 2 * a + n * b -
         m * floor_sum(n, a, b, m);
}
```

6.5 GCD LCM свёртки

```
vector<int> gcd_convolution(vector<int> a, vector<int> b) {
  // a,b is 1-indexed array; a[0],b[0] doesnt matter
  int n = ssize(a) - 1;
  for (int p = 2; p <= n; ++p) {
    if (!isprime[p]) continue; // alt: for (auto p : primes)
     if (p>n)break;
    for (int i = n / p; i > 0; --i) {
      a[i] += a[i * p];
      if (a[i] >= mod) a[i] -= mod;
      b[i] += b[i * p];
      if (b[i] >= mod) b[i] -= mod;
    }
  }
  for (int i = 1; i <= n; ++i) a[i] = (a[i] * b[i]) % mod;
  for (int p = 2; p <= n; ++p) {
    if (!isprime[p]) continue;
    for (int i = 1; i * p <= n; ++i) {
      a[i] += mod - a[i * p];
      if (a[i] >= mod) a[i] -= mod;
    }
  }
  return a;
}

vector<int> lcm_convolution(vector<int> a, vector<int> b) {
  int n = ssize(a) - 1;
  for (int p = 2; p <= n; ++p) {
    if (!isprime[p]) continue;
    for (int i = 1; i * p <= n; ++i) {
      a[i * p] += a[i];
      if (a[i * p] >= mod) a[i * p] -= mod;
      b[i * p] += b[i];
      if (b[i * p] >= mod) b[i * p] -= mod;
    }
  }
  for (int i = 1; i <= n; ++i) a[i] = (a[i] * b[i]) % mod;
  for (int p = 2; p <= n; ++p) {
    if (!isprime[p]) continue;
    for (int i = n / p; i > 0; --i) {
      a[i * p] += mod - a[i];
      if (a[i * p] >= mod) a[i * p] -= mod;
    }
  }
  return a;
}
```

6.6 Interpolation

```
struct poly {
  int n = 0;
  vector<ll> a = {0};
  poly() = default;
  poly(vector<ll> a) : a(a), n((int)a.size()-1) {}
  poly(int n) : n(n), a(n+1, 0) {}

  ll evaluate(int x) {
    ll val = 0, y = 1;
    for (int i = 0; i <= n; i++) {
      val = (val + a[i] * y) % MOD;
      y = (y * x) % MOD;
    }
    return val;
  }
};

poly interpolate(int deg, vector<int> x, vector<int> y) {
  assert(x.size() >= deg + 1 && y.size() >= deg + 1);
  int n = deg + 1;
  vector<ll> xs(n), c(n), cur(n);
  for (int i = 0; i < n; i++) {
    xs[i] = (x[i] % MOD + MOD) % MOD;
    c[i] = (y[i] % MOD + MOD) % MOD;
  }
  for (int k = 0; k + 1 < n; k++) {
    for (int i = k + 1; i < n; i++) {
      c[i] = (c[i] - c[k] + MOD) * inv((xs[i] - xs[k] + MOD) %
       MOD) % MOD;
    }
  }
  poly res(deg);
  cur[0] = 1;
  for (int k = 0; k < n; k++) {
    for (int i = 0; i <= k; i++) {
      res.a[i] = (res.a[i] + c[k] * cur[i]) % MOD;
    }
    if (k + 1 == n) break;
    for (int i = k + 1; i >= 0; i--) {
      cur[i] = ((i ? cur[i - 1] : 0) + (i <= k ? MOD - cur[i]
     * xs[k] % MOD : 0)) % MOD;
    }
  }
  return res;
}
```

6.7 Min25

```
// Sum multiplicative f over [1..n], O(n^{3/4}/log n).
// Need f(p) = polynomial in p. Change D, POLY, sum_power(),
     f_prime_power().
// Default: f(n) = n, f(p) = p, f(p^e) = p^e.
// Prime count: D=0, POLY={1}, build(n), answer = prime_poly(n
     ).
// phi: POLY={-1,1}, f_prime_power=(pe-prev+mod)%mod.
// f_prime_power: any O(1) in [0, mod), e=1 must match POLY.

// ll mul(ll a, ll b) in [0, mod); a, b may be negative!

struct Min25 {
  // f(p) = sum POLY[d] * p^d. Add degrees if needed.
  const int D = 1;
  const vector<ll> POLY = {0, 1}; // id: p

  ll n, sq;
  vector<ll> val;
  vector<int> id_small, id_large;
  vector<vector<ll>> g; // g[d][i] = sum_{p<=val[i]} p^d
  vector<ll> primes;
  bool built = false;

  int id(ll x) {
    return x <= sq ? id_small[x] : id_large[n / x];
  }

  ll sum_power(int d, ll x) {
    x %= mod;
    if (d == 0) return x;
    if (d == 1) return mul(x, x + 1) * ((mod + 1) / 2) % mod;
    // Add interpolation here for larger D.
    assert(false);
  }

  ll prime_poly_at_index(int i) {
    ll res = 0;
    for (int d = 0; d <= D; ++d)
      res = (res + mul(POLY[d], g[d][i])) % mod;
    return res;
  }

  ll prime_poly(ll x) {
    if (x < 2) return 0;
    return prime_poly_at_index(id(x));
  }

  // Value f(p^e). Args are (p, e, p^e mod mod, p^{e-1} mod
     mod).
  ll f_prime_power(ll p, int e, ll pe, ll prev) {
    return pe;
  }

  void build(ll _n) {
    if (built) {
      assert(n == _n);
      return;
    }
    built = true;
    n = _n;
    val.clear();
    primes.clear();
    sq = sqrtl(n);
    while ((sq + 1) * (sq + 1) <= n) ++sq;
    while (sq * sq > n) --sq;
    id_small.assign(sq + 1, -1);
    id_large.assign(sq + 1, -1);

    for (ll l = 1, r; l <= n; l = r + 1) {
      ll x = n / l;
      r = n / x;
      int i = val.size();
      val.push_back(x);
      if (x <= sq) id_small[x] = i;
      else id_large[n / x] = i;
    }
```

```
  g.assign(D + 1, vector<ll>(val.size()));
  for (int d = 0; d <= D; ++d)
    for (int i = 0; i < (int)val.size(); ++i)
      g[d][i] = (sum_power(d, val[i]) - 1 + mod) % mod;

  for (ll p = 2; p <= sq; ++p) {
    if (g[0][id(p)] == g[0][id(p - 1)]) continue;
    primes.push_back(p);
    vector<ll> ppow(D + 1, 1);
    for (int d = 1; d <= D; ++d) ppow[d] = mul(ppow[d - 1],
   p);
    for (int i = 0; i < (int)val.size() && val[i] >= p * p;
   ++i) {
      int ip = id(val[i] / p);
      int before_p = id(p - 1);
      for (int d = 0; d <= D; ++d) {
        ll sub = mul(g[d][ip] - g[d][before_p], ppow[d]);
        g[d][i] = (g[d][i] - sub + mod) % mod;
      }
    }
  }
}

// Sum of f(x), x <= limit, all prime divisors of x are >=
   primes[pos].
ll dfs(ll limit, int pos) {
  ll last = pos ? primes[pos - 1] : 1;
  ll res = (prime_poly(limit) - prime_poly(last)) % mod;
  if (res < 0) res += mod;
  if (limit >= 2 && pos < (int)primes.size()) {
    for (int i = pos; i < (int)primes.size(); ++i) {
      ll p = primes[i];
      if ((__int128)p * p > limit) break;
      ll pe_int = p, pe_mod = p % mod, prev_mod = 1;
      for (int e = 1; (__int128)pe_int * p <= limit; ++e) {
        ll nxt_mod = mul(pe_mod, p);
        ll cur = f_prime_power(p, e, pe_mod, prev_mod);
        ll nxt = f_prime_power(p, e + 1, nxt_mod, pe_mod);
        res = (res + mul(cur, dfs(limit / pe_int, i + 1)) +
   nxt) % mod;
        prev_mod = pe_mod;
        pe_mod = nxt_mod;
        pe_int *= p;
      }
    }
  }
  return res;
}

// After build(n), returns sum_{i<=x} f(i).
// Requires x <= sqrt(n) or x = floor(n / k) for some k.
ll prefix_sum(ll x) {
  if (x <= 0) return 0;
  assert(built && x <= n);
  if (x > sq) assert(val[id_large[n / x]] == x);
  return (1 + dfs(x, 0)) % mod; // f(1) = 1
}

// O(n^{3/4}/log n)
// Returns ans[id(x)] = sum_{i<=x} f(i) for every x in val.
vector<ll> all_prefix_sums() {
  assert(built);
  vector<ll> prime_sum(val.size()), r(val.size()), ans(val.
   size());
  for (int i = 0; i < (int)val.size(); ++i)
    prime_sum[i] = r[i] = prime_poly_at_index(i);
  for (int ui = (int)primes.size() - 1; ui >= 0; --ui) {
    ll p = primes[ui];
    ll before = prime_sum[id(p)];
    for (int i = 0; i < (int)val.size() && val[i] >= p * p;
   ++i) {
      ll pe_int = p, pe_mod = p % mod, prev_mod = 1;
      for (int e = 1; (__int128)pe_int * p <= val[i]; ++e) {
        ll nxt_mod = mul(pe_mod, p);
        ll cur = f_prime_power(p, e, pe_mod, prev_mod);
        ll nxt = f_prime_power(p, e + 1, nxt_mod, pe_mod);
        ll tail = r[id(val[i] / pe_int)] - before;
        if (tail < 0) tail += mod;
        r[i] = (r[i] + mul(cur, tail) + nxt) % mod;
        prev_mod = pe_mod;
        pe_mod = nxt_mod;
        pe_int *= p;
      }
    }
  }

  for (int i = 0; i < (int)val.size(); ++i)
    ans[i] = (1 + r[i]) % mod; // f(1) = 1
  return ans;
}

ll solve(ll _n) {
  if (_n <= 0) return 0;
  build(_n); // After build(n), prime_poly(n) is sum f(p)
   over primes p <= n.
  return prefix_sum(n);
}
};
```

6.8 NTT

```
const int MOD = 998244353; // 7 · 17 · 2^23 + 1
const int G = 3;
//const int MOD = 7340033; // 7 · 2^20 + 1
//const int G = 5;
//const int MOD = 469762049; // 7 · 2^26 + 1
//const int G = 30;
const int MAXLOG = 23;
int W[(1 << MAXLOG) + 10];
bool nttinit = false;
vector<int> pws;

// int add(), int sub(), int mul(),
// int binpow(), int inv()

void initNTT() {
  if (nttinit) return;
  nttinit = true;
  assert((MOD - 1) % (1 << MAXLOG) == 0);
  pws.push_back(binpow(G, (MOD - 1) / (1 << MAXLOG)));
  for (int i = 0; i < MAXLOG - 1; ++i)
    pws.push_back(mul(pws.back(), pws.back()));
  assert(pws.back() == MOD - 1);
  W[0] = 1;
  for (int i = 1; i < (1 << MAXLOG); ++i)
    W[i] = mul(W[i - 1], pws[0]);
}

void ntt(int n, vector<int> &a, bool rev) {
  initNTT();
  int lg = 31 - __builtin_clz(n);
  for (auto &el : a) {
    el = (el % MOD + MOD) % MOD;
  }
  vector<int> rv(n);
  for (int i = 1; i < n; ++i) {
    rv[i] = (rv[i >> 1] >> 1) ^ ((i & 1) << (lg - 1));
    if (rv[i] > i) swap(a[i], a[rv[i]]);
  }
  int num = MAXLOG - 1;
  for (int len = 1; len < n; len *= 2, --num) {
    for (int i = 0; i < n; i += 2 * len) {
      for (int j = 0; j < len; ++j) {
        int u = a[i + j];
        int v = mul(W[j << num], a[i + j + len]);
        a[i + j] = add(u, v);
        a[i + j + len] = sub(u, v);
      }
    }
  }
  if (rev) {
    int invn = binpow(n, MOD - 2);
    for (int i = 0; i < n; ++i) a[i] = mul(a[i], invn);
    reverse(a.begin() + 1, a.end());
  }
}

vector<int> conv(vector<int> a, vector<int> b) {
  if (a.empty() || b.empty())
    return {};
  int lg = 32-__builtin_clz(max<int>(1,a.size()+b.size()-2));
  int n = 1 << lg;
  a.resize(n);
  b.resize(n);
  ntt(n, a, false);
  ntt(n, b, false);
  for (int i = 0; i < n; ++i)
    a[i] = mul(a[i], b[i]);
  ntt(n, a, true);
  while (a.size() > 1 && a.back() == 0)
    a.pop_back();
  return a;
}

vector<int> add(vector<int> a, vector<int> b) {
  a.resize(max(a.size(), b.size()));
  for (int i = 0; i < (int) b.size(); ++i)
    a[i] = add(a[i], b[i]);
  return a;
}

vector<int> sub(vector<int> a, vector<int> b) {
  a.resize(max(a.size(), b.size()));
  for (int i = 0; i < (int) b.size(); ++i)
    a[i] = sub(a[i], b[i]);
  return a;
}

vector<int> inv(const vector<int> &a, int need) {
  vector<int> b = {inv(a[0])};
  while ((int) b.size() < need) {
    vector<int> a1 = a;
    int m = b.size();
    a1.resize(min((int) a1.size(), 2 * m));
    b = conv(b, sub({2}, conv(a1, b)));
    b.resize(2 * m);
  }
  b.resize(need);
  return b;
}
```

```
vector<int> div(vector<int> a, vector<int> b) {
  if (count(all(a), 0) == a.size())
    return {0};
  assert(a.back() != 0 && b.back() != 0);
  int n = a.size() - 1;
  int m = b.size() - 1;
  if (n < m)
    return {0};
  reverse(all(a));
  reverse(all(b));
  a.resize(n - m + 1);
  b.resize(n - m + 1);
  vector<int> c = inv(b, b.size());
  vector<int> q = conv(a, c);
  q.resize(n - m + 1);
  reverse(all(q));
  return q;
}

vector<int> mod(vector<int> a, vector<int> b) {
  auto res = sub(a, conv(b, div(a, b)));
  while (res.size() > 1 && res.back() == 0)
    res.pop_back();
  return res;
}

vector<int> multipoint(vector<int> a, vector<int> x) {
  int n = x.size();
  vector<vector<int>> tree(2 * n);
  for (int i = 0; i < n; ++i)
    tree[i + n] = {x[i], MOD - 1};
  for (int i = n - 1; i; --i)
    tree[i] = conv(tree[2 * i], tree[2 * i + 1]);
  tree[1] = mod(a, tree[1]);
  for (int i = 2; i < 2 * n; ++i)
    tree[i] = mod(tree[i >> 1], tree[i]);
  vector<int> res(n);
  for (int i = 0; i < n; ++i)
    res[i] = tree[i + n][0];
  return res;
}

vector<int> deriv(vector<int> a) {
  for (int i = 1; i < (int) a.size(); ++i)
    a[i - 1] = mul(i, a[i]);
  a.back() = 0;
  if (a.size() > 1)
    a.pop_back();
  return a;
}

vector<int> integ(vector<int> a) {
  a.push_back(0);
  for (int i = (int) a.size() - 1; i; --i)
    a[i] = mul(a[i - 1], inv(i));
  a[0] = 0;
  return a;
}

vector<int> log(vector<int> a, int n) {
  assert(a[0] == 1);
  auto res = integ(conv(deriv(a), inv(a, n)));
  res.resize(n);
  return res;
}

vector<int> exp(vector<int> a, int need) {
  assert(a[0] == 0);
  vector<int> b = {1};
  while ((int) b.size() < need) {
    vector<int> a1 = a;
    int m = b.size();
    a1.resize(min((int) a1.size(), 2 * m));
    a1[0] = add(a1[0], 1);
    b = conv(b, sub(a1, log(b, 2 * m)));
    b.resize(2 * m);
  }
  b.resize(need);
  return b;
}

vector<int> power_sum_poly(int k) {
  vector<int> a(k + 1), p(k + 2);
  for (int j = 0; j <= k; ++j)
    a[j] = ifact[j + 1];
  auto c = inv(a, k + 1);
  for (int j = 0; j <= k; ++j) {
    int v = mul(fact[k], mul(c[j], ifact[k + 1 - j]));
    p[k + 1 - j] = (j & 1) ? sub(0, v) : v;
  }
  return p; // P(n) = 1^k+ ... +n^k = sum p[i] n^i
}

vector<int> power_sums_inv(int n, int k) {
  vector<int> a(k + 1), b(k + 1);
  int pw = 1, t = add(n % MOD, 1);
  for (int i = 0; i <= k; ++i) {
    pw = mul(pw, t);
    a[i] = mul(sub(pw, 1), ifact[i + 1]);
    b[i] = ifact[i + 1];
  }
  auto s = conv(a, inv(b, k + 1));
  s.resize(k + 1);
  for (int i = 0; i <= k; ++i)
    s[i] = mul(s[i], fact[i]);
  return s; // s[i]=1^i+...+n^i; O(k log k)
}
```

6.9 OR XOR AND свёртки

```
vector<int> or_conv(int n, vector<int> a, vector<int> b) { // |a| = |b| = 2^n
  for (int i = 0; i < n; i++) {
    for (int j = 0; j < (1 << n); j++) {
      if ((j >> i) & 1) {
        a[j] = (a[j] + a[j ^ (1 << i)]) % MOD;
        b[j] = (b[j] + b[j ^ (1 << i)]) % MOD;
      }
    }
  }
  vector<int> c(1 << n);
  for (int i = 0; i < (1 << n); i++) {
    c[i] = (a[i] * b[i]) % MOD;
  }
  for (int i = n - 1; i >= 0; i--) {
    for (int j = (1 << n) - 1; j >= 0; j--) {
      if ((j >> i) & 1) {
        c[j] = (c[j] - c[j ^ (1 << i)] + MOD) % MOD;
      }
    }
  }
  return c;
}

vector<int> and_conv(int n, vector<int> a, vector<int> b) { // |a| = |b| = 2^n
  for (int i = 1; i < (1 << n); i *= 2) {
    for (int j = 0; j < (1 << n); j += i * 2) {
      for (int k = 0; k < i; k++) {
        a[j + k] = (a[j + k] + a[i + j + k]) % MOD;
        b[j + k] = (b[j + k] + b[i + j + k]) % MOD;
      }
    }
  }
  vector<int> c(1 << n);
  for (int i = 0; i < (1 << n); i++)
    c[i] = (a[i] * b[i]) % MOD;
  for (int i = 1; i < (1 << n); i *= 2) {
    for (int j = 0; j < (1 << n); j += i * 2) {
      for (int k = 0; k < i; k++) {
        c[j + k] = (c[j + k] - c[i + j + k] + MOD) % MOD;
      }
    }
  }
  return c;
}

const int inv2 = (MOD + 1) / 2;
vector<int> xor_conv(int n, vector<int> a, vector<int> b) { // |a| = |b| = 2^n
  for (int i = 1; i < (1 << n); i *= 2) {
    for (int j = 0; j < (1 << n); j++) {
      if ((j & i) == 0) {
        int x = a[j], y = a[j | i];
        a[j] = (x + y) % MOD, a[j | i] = (x - y + MOD) % MOD;
        x = b[j], y = b[j | i];
        b[j] = (x + y) % MOD, b[j | i] = (x - y + MOD) % MOD;
      }
    }
  }
  vector<int> c(1 << n);
  for (int i = 0; i < (1 << n); i++)
    c[i] = (a[i] * b[i]) % MOD;
  for (int i = 1; i < (1 << n); i *= 2) {
    for (int j = 0; j < (1 << n); j++) {
      if ((j & i) == 0) {
        int x = c[j], y = c[j | i];
        c[j] = (inv2 * (x + y)) % MOD, c[j | i] = (inv2 * (x -
      y + MOD)) % MOD;
      }
    }
  }
  return c;
}
```

6.10 convMod

```
// a[i] in [0, MOD), MOD <~ 1e9, n + m <= 2^20 (precision)
vector<ll> convMod(const vector<ll> &a, const vector<ll> &b) {
  if (a.empty() || b.empty()) return {};
  vector <ll> res((int) a.size() + (int) b.size() - 1);
  int lg = 32 - __builtin_clz(max<int>(1, res.size() - 1));
  int n = 1 << lg, cut = (int) (sqrt(MOD));
  vector <comp> L(n), R(n), outs(n), outl(n);
  // x = h * cut + l, |h|, |l| <~ cut / 2: 4x less fft error
  auto sp = [&](ll x) {
    if (x > MOD / 2) x -= MOD;
    ll h = rint((double)x / cut);
    return comp(h, x - h * cut);
```

```
  };
  for (int i = 0; i < a.size(); i++) L[i] = sp(a[i]);
  for (int i = 0; i < b.size(); i++) R[i] = sp(b[i]);
  fft(L, lg), fft(R, lg);
  for (int i = 0; i < n; i++) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + L[j].conj()) * R[i] / (2.0 * n);
    outs[j] = (L[i] - L[j].conj()) * R[i] / (2.0 * n) * comp
     (0, 1) * -1;
  }
  fft(outl, lg), fft(outs, lg);
  for (int i = 0; i < res.size(); i++) {
    ll av = rint(outl[i].x), cv = rint(outs[i].y);
    ll bv = rint(outl[i].y) + rint(outs[i].x);
    res[i] = ((av % MOD * cut + bv) % MOD * cut + cv) % MOD;
    res[i] += res[i] < 0 ? MOD : 0;
  }
  return res;
}
```

6.11 sqrt mod

```
// p is prime
// -1 if no solution
// x = sqrt(a,p) ⟹ x^2 = a and (-x)^2 = a
// O(log n) if p ≡ 3 mod 4 else O(log^2 n)
// should be changed if const p
ll sqrt(ll a, ll p) {
  a %= p;
  if (a < 0) a += p;
  if (a == 0) return 0;
  if (binpow(a, (p - 1) / 2, p) != 1)
    return -1; // no solution
  if (p % 4 == 3) return binpow(a, (p + 1) / 4, p);
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0) ++r, s /= 2;
  while (binpow(n, (p - 1) / 2, p) != p - 1) ++n;
  ll x = binpow(a, (s + 1) / 2, p);
  ll b = binpow(a, s, p), g = binpow(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m) t = t * t % p;
    if (m == 0) return x;
    ll gs = binpow(g, 1LL << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

6.12 Гаусс

```
// for modular: dbl -> ll, EPS=0
char sign(dbl x) { return (x>EPS) - (x<-EPS); }

// for double: ll -> dbl, no % mod, inv(x) = 1/x
vector<vector<ll>> gauss(vector<vector<ll>> &a) {
  int n = a.size();
  int m = a[0].size();
//  ll det = 1;
  for (int col = 0, row = 0; col < m && row < n; ++col) {
    int sel = row; // max |a| is for dbl precision
    for (int i = row; i < n; ++i) {
      if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
    }
    if (!sign(a[sel][col])) continue;
    swap(a[sel], a[row]);
//    if (sel != row) det *= -1;
    ll iv = inv(a[row][col]);
    for (int i = 0; i < n; ++i) {
      if (i != row && sign(a[i][col])) {
        ll val = a[i][col] * iv % mod;
        for (int j = col; j < m; ++j) {
          a[i][j] -= val * a[row][j];
          a[i][j] %= mod;
        }
      }
    }
    ++row;
  }
//  for (int i = 0; i < n; ++i) det = (det * a[i][i]) % mod;
//  det = (det % mod + mod) % mod;
// result in (-mod, mod)
  return a;
}

pair<int,vector<ll>> sle(vector<vector<ll>> a, vector<ll> b) {
  int n = a.size();
  int m = a[0].size();
  assert(n == b.size());
  for (int i = 0; i < n; ++i) {
    a[i].push_back(b[i]);
  }
  a = gauss(a);
  vector<ll> x(m, 0);
  for (int i = n - 1; i >= 0; --i) {
    int leftmost = m;
    for (int j = 0; j < m; ++j) {
      if (sign(a[i][j])) {
        leftmost = j;
        break;
      }
    }
    if (leftmost == m && sign(a[i].back())) return {-1, {}};
    if (leftmost == m) continue;
    ll val = a[i].back();
    for (int j = m - 1; j > leftmost; --j) {
      val -= a[i][j] * x[j];
      val %= mod;
    }
    x[leftmost] = (val * inv(a[i][leftmost]) % mod + mod) %
     mod;
  }
  return {1, x};
}

vector<bitset<N>> gauss_bit(vector<bitset<N>> a, int m) {
  int n = a.size();
  for (int col = 0, row = 0; col < m && row < n; ++col) {
    for (int i = row; i < n; ++i) {
      if (a[i][col]) {
        swap(a[i], a[row]);
        break;
      }
    }
    if (!a[row][col]) continue;
    for (int i = 0; i < n; ++i)
      if (i != row && a[i][col]) a[i] ^= a[row];
    ++row;
  }
  return a;
}
```

6.13 Диофантовы уравнения

```
// ax + by = ±gcd if a < 0 or b < 0
pair<int, int> ext_gcd(int a, int b) {
  int x1 = 1, y1 = 0, x2 = 0, y2 = 1;
  while (b) {
    int k = a / b;
    x1 -= x2 * k;
    y1 -= y2 * k;
    a %= b;
    swap(x1, x2), swap(y1, y2), swap(a, b);
  }
  return {x1, y1};
}

// solve ax + by = c with minimum x ≥ 0
bool cool_ext_gcd(int a, int b, int c, int &x, int &y) {
    if (b == 0) {
        y = 0;
        if (a == 0)
            return x = 0, c == 0;
        return x = c / a, c % a == 0 && x >= 0;
    }
    auto [x0, y0] = ext_gcd(a, b);
    int g = (ll)x0 * a + (ll)y0 * b;
    if (c % g != 0) return false;
    x = (ll)x0 * (c / g) % (b / g);
    if (x < 0) x += abs(b / g);
    y = (c - (ll)a * x) / b;
    return true;
}
```

6.14 КТО

```
// solve w ≡ r_i (mod c_i)
// assumes c_i > 0 and lcm(c_i) fits in ll
// return minimum w ≥ 0 and common modulo
// return {-1, -1} if there is no solution
pair<ll, ll> generalized_crt(const vector<pair<int, int>> &
    equations) {
  ll r = 0, mod = 1;
  for (auto [ri, ci] : equations) {
    ri %= ci;
    if (ri < 0) ri += ci;
    int reduced_mod = mod % ci;
    auto [p, q] = ext_gcd(reduced_mod, ci);
    int g = (ll)p * reduced_mod + (ll)q * ci;
    ll diff = ri - r;
    if (diff % g != 0) return {-1, -1};
    ll k = (__int128)(diff / g) * p % (ci / g);
    if (k < 0) k += ci / g;
    ll new_mod = mod / g * ci;
    r = (r + (__int128)mod * k) % new_mod;
    mod = new_mod;
  }
  return {r, mod};
}
```

6.15 Код Грея

```
for (int i = 0; i < (1 << n); i++) {
  gray[i] = i ^ (i >> 1);
}
```

6.16 Линейное решето

```
const int N = 10000000;
int lp[N + 1];
vector<int> pr;
for (int i = 2; i <= N; ++i) {
  if (lp[i] == 0) {
    lp[i] = i;
    pr.push_back(i);
  }
  for (int j = 0; j < (int) pr.size() && pr[j] <= lp[i] && i *
      pr[j] <= N; ++j)
    lp[i * pr[j]] = pr[j];
}
```

6.17 Миллер Рабин

```
// works for all n < 2^64
const ll MAGIC[7] = {2, 325, 9375, 28178, 450775, 9780504,
    1795265022};

bool is_prime(ll n) {
  if (n <= 1) return false;
  if (n <= 3) return true;
  if (n % 2 == 0 || n % 3 == 0) return false;
  ll s = __builtin_ctzll(n - 1), d = n >> s; // n - 1 = 2^s * d
  for (auto a : MAGIC) {
    if (a % n == 0) {
      continue;
    }
    ll x = binpow(a, d, n); // a -> __int128 in binpow
    for (int _ = 0; _ < s; _++) {
      ll y = binpow(x, 2, n);  // x -> __int128 in binpow
      if (y == 1 && x != 1 && x != n - 1) {
        return false;
      }
      x = y;
    }
    if (x != 1) {
      return false;
    }
  }
  return true;
}
```

6.18 Подсчёт прогулок

```
int count_walks_1(int b1, int b2, int p, int q) {
  // counting walks from (0, 0) to (p, q)
  // each turn x += 1 or y += 1
  // without touching y = x + b1 and y = x + b2
  // b1 < 0 < b2 must hold
  // O((p + q) / (b2 - b1))
  if (min(p, q) < 0 || q - p <= b1 || q - p >= b2) return 0;
  ll ans = C(p, p + q);
  ar(2) F = {p, q}, S = {p, q};
  int cf = mod - 1;
  while (true) {
    F[1] -= b1;
    swap(F[0], F[1]);
    F[1] += b1;
    S[1] -= b2;
    swap(S[0], S[1]);
    S[1] += b2;
    swap(F, S);
    int wf = C(F[0], F[0] + F[1]);
    int ws = C(S[0], S[0] + S[1]);
    ans += (cf * (ll) ((wf + ws) % mod)) % mod;
    if (wf == 0 && ws == 0) break;
    cf = mod - cf;
  }
  ans %= mod;
  return (int) ans;
}

int count_walks_2(int b1, int b2, int p, int q) {
  // counting walks from (0, 0) to (p, q)
  // each turn x += 1 and (y -= 1 or y += 1)
  // without touching y = b1 and y = b2
  // b1 < 0 < b2 must hold
  // O(p / (b2 - b1))
  if (abs(p) % 2 != abs(q) % 2) return 0;
  int p0 = (p - q) / 2, q0 = (p + q) / 2;
  return count_walks_1(b1, b2, p0, q0);
}
```

6.19 Ро-Поллард

```
typedef long long ll;

ll mult(ll a, ll b, ll mod) {
  return (__int128)a * b % mod;
}

ll f(ll x, ll c, ll mod) {
  return (mult(x, x, mod) + c) % mod;
}

ll rho(ll n, ll x0=2, ll c=1) {
  ll x = x0;
  ll y = x0;
  ll g = 1;
  while (g == 1) {
    x = f(x, c, n);
    y = f(y, c, n);
    y = f(y, c, n);
    g = gcd(abs(x - y), n);
  }
  return g;
}

mt19937_64 rnd(time(nullptr));

void factor(int n, vector<int> &pr) {
  if (n == 4) {
    factor(2, pr);
    factor(2, pr);
    return;
  }
  if (n == 1) {
    return;
  }
  if (is_prime(n)) {
    pr.push_back(n);
    return;
  }
  int d = rho(n, rnd() % (n - 2) + 2, rnd() % 3 + 1);
  factor(n / d, pr);
  factor(d, pr);
}
```

6.20 Чудо Формулы

1. $\sum_{0\le k\le n}\binom{n-k}{k} = \mathrm{Fib}_{n+1}$
2. $\sum_{i=0}^{k}(-1)^i\binom{n}{i} = (-1)^k\binom{n-1}{k}$
3. $\sum_{i=0}^{k}\binom{n+i}{i} = \binom{n+k+1}{k}$
4. $\sum_{k=0}^{r}\binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$
5. $\sum_{i=r}^{n}\binom{i}{r} = \binom{n+1}{r+1}$
6. **Сдвинутый биномиальный базис**:
   $\Phi_{i,d}(x) = [x \ge i]\binom{x-i+d-1}{d-1}, \quad d \ge 1,\ i \in \mathbb{Z}$
   $F(x) = \sum_{i=1}^{d} c_i\Phi_{i,d}(x), \quad c_i = 0\ (i \notin [1,d])$
   $\texttt{lift}: \quad \Phi_{i,d} = \Phi_{i,d+1} - \Phi_{i+1,d+1}; \quad \overline{c}_i = c_i - c_{i-1}; \quad \overline{d} = d+1$
   $A_i = i + \lambda, \qquad B_i = d - i - \lambda$
   $(x+\lambda)\Phi_{i,d} = A_i\Phi_{i,d+1} + B_i\Phi_{i+1,d+1}; \quad \overline{c}_i = A_ic_i + B_{i-1}c_{i-1}; \quad \overline{d} = d+1$
   $\sum_{t=i}^{x-k}\Phi_{i,d}(t)\Phi_{k,r}(x-t) = \Phi_{i+k,d+r}(x)$
   $\sum_{t=1}^{x-k}F(t)\Phi_{k,r}(x-t) \Longrightarrow \overline{c}_i = c_{i-k}, \quad \overline{d} = d+r$
   $\texttt{prefsum}^s: \quad \sum_{t=1}^{x}F(t)\binom{x-t+s-1}{s-1} \Longrightarrow \overline{c}_i = c_i, \quad \overline{d} = d+s$
   $\texttt{shiftsum}: \quad \sum_{t=1}^{x-1}F(t) \Longrightarrow \overline{c}_i = c_{i-1}, \quad \overline{d} = d+1$
   $\texttt{shiftsum; prefsum}^s: \quad \sum_{t=1}^{x}F(t)\binom{x-t+s-1}{s} \Longrightarrow \overline{c}_i = c_{i-1}, \quad \overline{d} = d+s+1$
   $F+G, \quad d_F = d_G: \qquad c_i = c_i^F + c_i^G$
   $F\cdot G: \quad D = d_F + d_G - 1, \qquad h_d(r) = [r \ge 0]\binom{r+d-1}{d-1}$
   $f_x = \sum_{i=1}^{d_F}c_i^F h_{d_F}(x-i), \quad g_x = \sum_{i=1}^{d_G}c_i^G h_{d_G}(x-i), \quad p_x = f_xg_x \qquad (1 \le x \le D)$
   $\overline{c}_i = \sum_{j=0}^{i-1}(-1)^j\binom{D}{j}p_{i-j}, \qquad \overline{d} = D \quad (1 \le i \le D)$
   $f,\ g,\ \overline{c}$ считаются тремя NTT-свёртками
7. Число решений $x_1 + \cdots + x_n = N,\ 0 \le x_i \le r$:
   $$\sum_{q=0}^{\min(n,\lfloor N/(r+1)\rfloor)}(-1)^q\binom{n}{q}\binom{N-q(r+1)+n-1}{n-1}$$
8. $2B(l,r,n) = \binom{n}{l} + \binom{n}{r} - \binom{n+1}{l} + B(l,r,n+1)$, где $B(l,r,n) = \sum_{i=l}^{r}\binom{n}{i}$
9. Степень p в $\binom{n}{m}$ равна числу переносов при сложении m и $n-m$ в p-ичной системе.
10. $\sum_{i=0}^{n}\frac{\binom{k}{i}}{\binom{n}{i}} = \frac{\binom{n+1}{n-k+1}}{\binom{n}{k}}$
11. Число перестановок без фиксированных точек:
    $d(n) = (n-1)\big(d(n-1) + d(n-2)\big),\ d(0) = 1,\ d(1) = 0$
12. $\binom{m}{n} \equiv \prod_i\binom{m_i}{n_i} \pmod p$, где m_i, n_i — цифры m, n в p-ичной записи.
13. $\sum_{i=0}^{n}\binom{n}{i}i^k = \sum_{j=0}^{k}s_2(k,j)\,\frac{n!}{(n-j)!}\,2^{n-j}$
14. $\sum_{i=0}^{n-1}\binom{i}{j}x^i = x^j(1-x)^{-j-1}\Big(1 - x^{n-j}\sum_{i=0}^{j}\binom{n}{i}x^{j-i}(1-x)^i\Big)$
15. $P(n) = \sum_{k=0}^{n}\binom{n}{k}Q(k) \implies Q(n) = \sum_{k=0}^{n}(-1)^{n-k}\binom{n}{k}P(k)$
16. $P(n) = \sum_{k=0}^{n}(-1)^k\binom{n}{k}Q(k) \implies Q(n) = \sum_{k=0}^{n}(-1)^k\binom{n}{k}P(k)$
17. $N(n,k)$ — число ПСП длины 2n с 2k блоками $N(n,k) = \binom{n}{k-1}\cdot\binom{n}{k}/n$
18. $(-1)^{n-k}s_1(n,k)$ — число перестановок длины n с k циклами,
    $s_1(n,k) = s_1(n-1,k-1) - (n-1)s_1(n-1,k), \quad s_1(0,0) = 1$

19. $s_2(n,k)$ — число разбиений n-множества на k непустых блоков, $s_2(n,k) = k s_2(n-1,k) + s_2(n-1,k-1), \quad s_2(0,0) = 1$
20. $x(x-1)(x-2)\cdots(x-n+1) = \sum_{k=0}^{n} s_1(n,k) x^k$
21. $s_1(n,k) = \frac{n!}{k!}[x^{n-k}] \exp\left(k \log\left(\sum_{t\geq 0} \frac{(-1)^t x^t}{t+1}\right)\right)$
22. $s_2(n,k) = [x^{n-k}] \frac{1}{(1-x)(1-2x)\cdots(1-kx)}$
23. $s_2(n,k) = [x^k]\left(\sum_{i=0}^{n} \frac{i^n}{i!} x^i\right)\left(\sum_{j=0}^{n} \frac{(-1)^j}{j!} x^j\right)$
24. $P(k) = \sum_{i=1}^{k} s_2(k,i) Q(i) \implies Q(k) = \sum_{i=1}^{k} s_1(k,i) P(i)$
25. $s_2^d(n,k)$ — число разбиений с расстоянием $\geq d$ внутри блока, $s_2^d(n,k) = s_2(n-d+1, k-d+1)$
26. $\sum_{i=1}^{n} i a^i = \frac{a\left(n a^{n+1} - (n+1)a^n + 1\right)}{(a-1)^2}$
27. $e^x = \sum_{k\geq 0} \frac{x^k}{k!}, \quad \log(1+x) = \sum_{k\geq 1} (-1)^{k+1} \frac{x^k}{k}, \quad -\log(1-x) = \sum_{k\geq 1} \frac{x^k}{k}$
28. $(1+x)^\alpha = \sum_{k\geq 0} \binom{\alpha}{k} x^k, \quad \binom{\alpha}{k} = \frac{\alpha(\alpha-1)\cdots(\alpha-k+1)}{k!}$
    $(1-x)^{-n} = \sum_{k\geq 0} \binom{n+k-1}{n-1} x^k$
29. $\sin x = \sum_{k\geq 0} (-1)^k \frac{x^{2k+1}}{(2k+1)!}, \quad \cos x = \sum_{k\geq 0} (-1)^k \frac{x^{2k}}{(2k)!}$
    $\arctan x = \sum_{k\geq 0} (-1)^k \frac{x^{2k+1}}{2k+1} \quad (|x| \leq 1)$
30. $\prod_{n=1}^{\infty}(1-x^n) = 1 + \sum_{k=1}^{\infty} (-1)^k \left(x^{k(3k-1)/2} + x^{k(3k+1)/2}\right)$
31. n — Фибоначчи $\iff$ хотя бы одно из чисел $5n^2 \pm 4$ — точный квадрат.
32. Период последовательности Фибоначчи по модулю n не превышает $6n$.
33. Пифагоровы тройки: пусть $m > n > 0$, $\gcd(m,n) = 1$, и не оба нечётные. Тогда $a = k(m^2 - n^2)$, $b = k(2mn)$, $c = k(m^2 + n^2)$
34. $\#\{(a,b) : a^2 + b^2 = n^2, a > 0, b > 0\} = \left(\prod_{p^\alpha \| n,\ p=4k+1} 2\alpha + 1\right) - 1$
35. $r_4(n) = \#\{x_1^2 + \cdots + x_4^2 = n\}$, $r_8(n) = \#\{x_1^2 + \cdots + x_8^2 = n\}$
    $r_4(n) = 8 \sum_{d|n, 4\nmid d} d$, $r_8(n) = 16 \sum_{d|n} (-1)^{n+d} d^3$
36. $\#\{ax + by = n,\ x, y \geq 0\} = \frac{n}{ab} - \left\{\frac{b'n}{a}\right\} - \left\{\frac{a'n}{b}\right\} + 1,$
    где a' и b' — обратные: $aa' \equiv 1 \pmod b$, $bb' \equiv 1 \pmod a$
37. $\varphi(mn) = \varphi(m)\varphi(n)\frac{d}{\varphi(d)}$, $d = \gcd(m,n)$
38. $n^x \bmod m = n^{\varphi(m) + (x \bmod \varphi(m))} \bmod m$, $x \geq \log_2 m$
39. $\sum_{n=1}^{N} \mu^2(n) = \sum_{k=1}^{\lfloor\sqrt{N}\rfloor} \mu(k) \lfloor N/k^2 \rfloor$
40. $g(n) = \sum_{d|n} f(d) \implies f(n) = \sum_{d|n} \mu(d)\, g(n/d)$
41. $F(n) = \prod_{d|n} f(d) \implies f(n) = \prod_{d|n} F(n/d)^{\mu(d)}$
42. $\gcd\big(a, \operatorname{lcm}(b,c)\big) = \operatorname{lcm}\big(\gcd(a,b), \gcd(a,c)\big)$
43. $\operatorname{lcm}\big(a, \gcd(b,c)\big) = \gcd\big(\operatorname{lcm}(a,b), \operatorname{lcm}(a,c)\big)$
44. $\gcd\big(\operatorname{lcm}(a,b), \operatorname{lcm}(b,c), \operatorname{lcm}(a,c)\big) =$
    $= \operatorname{lcm}\big(\gcd(a,b), \gcd(b,c), \gcd(a,c)\big)$
45. $\sum_{i,j=1}^{n} \operatorname{lcm}(i,j) = \sum_{l=1}^{n} \left(\frac{(1+\lfloor n/l \rfloor)\lfloor n/l \rfloor}{2}\right)^2 \sum_{d|l} \mu(d) l d$
46. $\sum_{i=1}^{n} \operatorname{lcm}(i,n) = \frac{n}{2}\left(\sum_{d|n} \varphi(d) d + 1\right)$
47. $\left(\frac{p}{q}\right)\left(\frac{q}{p}\right) = (-1)^{\frac{(p-1)(q-1)}{4}}$ для нечётных простых p, q
48. $\varphi(m+1)$ — кол-во бинарных строк с m различными подпоследовательностями.
49. $S = I + \frac{B}{2} - 1$, S площадь, I целые точки внутри, B целые точки на границе.
50. $V - E + F = 2$, V вершины, E рёбра, F грани.
51. Целые $n > d_1 \geq \cdots \geq d_n \geq 0$ — степени простого неориентированного графа $\iff \sum_{i=1}^{n} d_i \equiv 0 \pmod 2$ и
    $\forall k = 1, \ldots, n: \quad \sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$
52. Простые числа
    - big NTT-friendly:
      - $29 \cdot 2^{57} + 1 \approx 4 \cdot 10^{18}$, $g = 3$
      - $27 \cdot 2^{56} + 1 \approx 2 \cdot 10^{18}$, $g = 5$
      - $237 \cdot 2^{52} + 1 \approx 1 \cdot 10^{18}$, $g = 5$
    - $= 10^{18}$ +3, +9, +31, +79, +177, +183, +201, +283
    - $= 10^{9}$ +7, +9, +21, +33, +87, +93, +97, +103, +123
    - 1791791791, 1791791797, 1791796969, 1234676767
    - 1000575757, 1000696969, 1000001791
    - 998696969, 998575757, 997676767, 996179179, 991676767

7 Строки

7.1 Z, префикс, Манакер, min rotation

```
vector<int> z_func(const string &s) {
  int n = s.size();
  vector<int> z(n, 0);
  if (n) z[0] = n;
  int l = 0, r = 0;
  for (int i = 1; i < n; i++) {
    if (i < r) z[i] = min(z[i-l], r-i);
    while (i+z[i] < n && s[z[i]] == s[i+z[i]]) z[i]++;
    if (i+z[i] > r) l = i, r = i+z[i];
  }
  return z;
}

vector<int> prefix_func(const string &s) {
  int n = s.size(), a = 0;
  vector<int> pref(n, 0);
  for (int i = 1; i < n; i++) {
    while (a > 0 && s[a] != s[i]) a = pref[a-1];
    pref[i] = a += s[i] == s[a];
  }
  return pref;
}

// d[0][i] even center (i,i+1). d[1][i] odd center (i)
// radius = ⌈len/2⌉
array<vector<int>, 2> manacher(const string &s) {
  int n = s.size();
  array<vector<int>, 2> d;
  if (!n) return d;
  for (int z = 0; z <= 1; ++z) { //0=odd,1=even here!
    d[z].resize(n - z);
    int l = 0, r = -1;
    for (int i = z; i < n; ++i) {
      int k = i > r ? 1-z : min(d[z][l+r-i], r-i+z);
      while(i-k-z>=0 && i+k<n && s[i-k-z]==s[i+k]) ++k;
      d[z][i-z] = k--;
      if (i+k > r) l = i-k-z, r = i+k;
    }
  }
  swap(d[0], d[1]);
  return d;
}

// лексикографичкски минимальный циклический сдвиг
int min_rotation(const string &s) {
  int a=0, n=s.size(); string t=s+s;
  for (int b=1; b<n; ++b)
    for (int k=0; k<n; ++k)
      if (a+k==b||t[a+k]<t[b+k]) { b+=k?k-1:0; break; }
      else if (t[a+k]>t[b+k]) { a=b; break; }
  return a;
}
```

7.2 eertree

```
int len[MAXN], suf[MAXN];
int go[MAXN][ALPH];
char s[MAXN];

int n, last, sz;

void init() {
  for (int v = 0; v < sz; v++) fill(go[v], go[v] + ALPH, 0);
  n = 0, last = 0;
  s[n++] = -1;
  suf[0] = 1; // root of suflink tree = 1
  len[1] = -1;
  sz = 2;
}

int get_link(int v) {
  while (s[n - len[v] - 2] != s[n - 1])
    v = suf[v];
  return v;
}

void add_char(char c) {
  c -= 'a';
  s[n++] = c;
  last = get_link(last);
  if (!go[last][c]) {
    len[sz] = len[last] + 2;
    suf[sz] = go[get_link(suf[last])][c];
    go[last][c] = sz++;
  }
  last = go[last][c]; // cur v = last
}
```

7.3 Ахо-Корасик

```
const int SIGMA = 26; // letters from 'a'
struct AhoCorasick {
  vector<array<int, SIGMA>> go;
  vector<int> par, suf;
  vector<char> par_c;
  vector<vector<int>> term, g; // g: suffix link tree
  int cntv = 1, cnt_s = 0;
  // maxn >= sum of lengths + 1
  AhoCorasick(int maxn) : go(maxn), par(maxn), suf(maxn),
      par_c(maxn), term(maxn), g(maxn) {}

  // returns node of s, term[v] = ids of strings (from 0)
  int add(const string &s) {
    int v = 0;
    for (char el : s) {
      if (go[v][el - 'a'] == 0) {
        go[v][el - 'a'] = cntv;
        par[cntv] = v;
        par_c[cntv] = el;
        cntv++;
      }
      v = go[v][el - 'a'];
    }
    term[v].pb(cnt_s++);
    return v;
  }

  void bfs() {
    vector<int> q(cntv); // q[0] = 0
    for (int l = 0, r = 1; l < r; ++l) {
      int v = q[l];
      if (v > 0) g[suf[v]].pb(v);
      for (int c = 0; c < SIGMA; c++) {
        int &u = go[v][c];
        if (u == 0) u = go[suf[v]][c];
        else suf[u] = v ? go[suf[v]][c] : 0, q[r++] = u;
      }
    }
  }
};
```

7.4 Муффиксный Сассив

```
template<class T> // string (<= 127 distinct chars) or vector
vector<int> build_suff_arr(T s) {
  { // values compression. comment if already 0 < s[i] < ~1e6
    vector<int> id(size(s));
    iota(all(id), 0);
    sort(all(id), [&](int i, int j) { return s[i]<s[j]; });
    auto v = s;
    for (int i=0, r=0; i < ssize(s); i++)
      v[id[i]] = r += !i || s[id[i]] != s[id[i-1]];
    s = v;
  }
  s.pb(0); // remove if cyclic shifts
  int n = s.size(), m = *max_element(all(s)) + 2;
  vector<int> sa(n), rk(n), t(n), c(max(n,m));
  for (auto x : s) c[x+1]++;
  partial_sum(all(c), begin(c));
  for (int i=0; i<n; i++) rk[i] = c[s[i]];
  for (int i=0; i<n; i++) sa[c[s[i]]++] = i;
  // rk[i] = start of i's class in sa
  for (int k=1, g=0; k<n && g<n; k*=2) {
    iota(all(c), 0);
    for (int i : sa) {
      int j = i>=k ? i-k : i-k+n;
      t[c[rk[j]]++] = j;
    }
    swap(sa,t), g=0;
    for (int i=0, pr=-1, pq=-1, st=0; i<n; i++) {
      int b=sa[i], j = b+k<n ? b+k : b+k-n;
      int r=rk[b], q=rk[j];
      if (r!=pr || q!=pq) st=i, g++;
      t[b]=st, pr=r, pq=q;
    }
    swap(rk,t);
  }
  sa.erase(sa.begin()); // remove if cyclic shifts
  return sa;
}

// lcp[i] = lcp(sa[i], sa[i + 1]), size = n-1
template<class T>
vector<int> kasai(const T &s, const vector<int> &sa) {
  int n = s.size(), k=0;
  vector<int> rk(n), lcp(n-!!n);
  for (int i=0; i<n; i++) rk[sa[i]] = i;
  for (int i=0; i<n; i++, k-=k>0) {
    if (rk[i] == n-1) continue;
    int j = sa[rk[i]+1];
    while (max(i,j)+k < n && s[i+k]==s[j+k]) k++;
    lcp[rk[i]] = k;
  }
  return lcp;
}
```

7.5 Суффиксный автомат

```
// (Обобщённый) суфавтомат для кол-ва различных подстрок
const int SIGMA = 26;
long long ans = 0;

struct Map{ //если большой алфавит, просто заменить ar на Map
  map<int, int> mp;
  int &operator[](int x) { return mp.try_emplace(x, -1).first
     ->second; }
};
struct Node {
  ar(SIGMA) go;
  int s, p, len, firstpos;
  Node() {
    go.fill(-1); // remove if Map
    s=-1, p=-1, len=0, firstpos=-1;
  }
};

int add(int A, int ch, vector<Node> &sa) {
  auto clone = [&](int A, int C) {
    int D = sa.size();
    sa.pb(sa[C]);
    sa[D].p = A;
    sa[D].len = sa[A].len + 1;
    sa[C].s = D;
    for (; A != -1 && sa[A].go[ch] == C; A = sa[A].s)
      sa[A].go[ch] = D;
    return D;
  };
  int C = sa[A].go[ch];
  if (C != -1)
    return sa[C].p == A ? C : clone(A, C);
  int B = sa.size();
  sa.emplace_back();
  sa[B].p = A;
  sa[B].len = sa[A].len + 1;
  sa[B].firstpos = sa[B].len - 1; // no sense when generalized
      SAM
  for (; A != -1; A = sa[A].s) {
    if (sa[A].go[ch] == -1) {
      sa[A].go[ch] = B;
      continue;
    }
    C = sa[A].go[ch];
    sa[B].s = sa[C].p == A ? C : clone(A, C);
    break;
  }
  if (A == -1) sa[B].s = 0;
  ans += sa[B].len - sa[sa[B].s].len;
  return B;
}

signed main() {
  string s;
  cin >> s;
  vector<Node> sa(1);
  int A = 0;
  for (char c : s)
    A = add(A, c - 'a', sa);
  cout << ans;
}
```

8 Структуры данных

8.1 Disjoint Sparse Table

```
// MAXN дополнить до степени двойки (или n*2)
// min -> любая ассоциативная операция (порядок важен)
int tree[LOG][MAXN];
int floorlog2[MAXN]; // i ? (31 - __builtin_clz(i)) : 0

void build(vector<int> &a) {
  int n = a.size();
  copy(all(a), tree[0]);
  for (int lg = 1; lg < LOG; ++lg) {
    int len = 1 << lg;
    auto &lvl = tree[lg];
    for (int m = len; m < n; m += len * 2) {
      lvl[m - 1] = a[m - 1];
      lvl[m] = a[m];
      for (int i = m - 2; i >= m - len; --i)
        lvl[i] = min(a[i], lvl[i + 1]);
      for (int i = m + 1; i < m + len && i < n; ++i)
        lvl[i] = min(lvl[i - 1], a[i]);
    }
  }
  for (int i = 2; i < min(MAXN, n * 2); ++i)
    floorlog2[i] = floorlog2[i / 2] + 1;
}

// a[l..r)
int get(int l, int r) {
  if (l == --r) return tree[0][l];
  int i = floorlog2[l ^ r];
  return min(tree[i][l], tree[i][r]);
}
```

8.2 Segment Tree Beats

```
// %=, =, sum
// mx[v], all_equal[v]
// break: mx[v] < x
// tag: all_equal[v] == true, запрос становится =mx[v]%x

// min=, max=, =, +=, sum, mn, mx
// также как и для min=, sum
// для max= храним mn[v], sec_mn[v]

// +=, gcd
// храним gcd разностей какого-то остовного дерева
// храним any_value[v] = любое значение на отрезке
// gcd(l...r) = gcd(any_value[v], gcd[v])
// при сливании добавляем к gcd значение |a_v[l] - a_v[r]|

// min=, sum
struct ST {
  vector<int> st, mx, mx_cnt, sec_mx;

  ST(vector<int> &a) {
    int n = a.size();
    st.resize(n * 4), mx.resize(n * 4);
    mx_cnt.resize(n * 4, 0), sec_mx.resize(n * 4, 0);
    build(0, 0, n, a);
  }

  void upd_from_children(int v) {
    st[v] = st[v * 2 + 1] + st[v * 2 + 2];
    mx[v] = max(mx[v * 2 + 1], mx[v * 2 + 2]);
    mx_cnt[v] = 0;
    sec_mx[v] = max(sec_mx[v * 2 + 1], sec_mx[v * 2 + 2]);
    if (mx[v * 2 + 1] == mx[v]) {
      mx_cnt[v] += mx_cnt[v * 2 + 1];
    } else {
      sec_mx[v] = max(sec_mx[v], mx[v * 2 + 1]);
    }
    if (mx[v * 2 + 2] == mx[v]) {
      mx_cnt[v] += mx_cnt[v * 2 + 2];
    } else {
      sec_mx[v] = max(sec_mx[v], mx[v * 2 + 2]);
    }
  }

  void build(int i, int l, int r, vector<int> &a) {
    if (l + 1 == r) {
      st[i] = mx[i] = a[l];
      mx_cnt[i] = 1;
      sec_mx[i] = -INF;
      return;
    }
    int m = (r + l) / 2;
    build(i * 2 + 1, l, m, a);
    build(i * 2 + 2, m, r, a);
    upd_from_children(i);
  }

  void push_min_eq(int v, int val) {
    if (mx[v] > val) {
      st[v] -= (mx[v] - val) * mx_cnt[v];
      mx[v] = val;
    }
  }

  void push(int i, int l, int r) {
    if (l + 1 < r) {
      push_min_eq(i * 2 + 1, mx[i]);
      push_min_eq(i * 2 + 2, mx[i]);
    }
  }

  void update(int i, int l, int r, int ql, int qr, int val) {
    if (qr <= l || r <= ql || mx[i] <= val) return;
    if (ql <= l && r <= qr && sec_mx[i] < val) {
      push_min_eq(i, val);
      return;
    }
    push(i, l, r);
    int m = (l + r) / 2;
    update(i * 2 + 1, l, m, ql, qr, val);
    update(i * 2 + 2, m, r, ql, qr, val);
    upd_from_children(i);
  }

  int sum(int i, int l, int r, int ql, int qr) {
    if (qr <= l || r <= ql) return 0;
    push(i, l, r);
    if (ql <= l && r <= qr) return st[i];
    int m = (l + r) / 2;
    return sum(i * 2 + 1, l, m, ql, qr) + sum(i * 2 + 2, m, r,
      ql, qr);
  }
};
```

8.3 ДД по неявному

```
// потому что nds[0].sz == 0 и sz не изменяется в push
int size(int t) { return nds[t].sz; }

pair<int, int> split(int t, int k) {
  if (!t) return {0, 0};
  push(t);
  int szl = size(nds[t].l);
  if (k <= szl) {
    auto [l, r] = split(nds[t].l, k);
    nds[t].l = r;
    pull(t);
    return {l, t};
  } else {
    auto [l, r] = split(nds[t].r, k - szl - 1);
    nds[t].r = l;
    pull(t);
    return {t, r};
  }
}

// всё остальное ровно как в обычном ДД
// не забыть обновлять sz в pull
// инициализация sz=0 в Node() и sz=1 в Node(...)
```

8.4 ДД

```
// insert (key, val), erase key, max(val) for key ∈ [l,r), val+=
    for key ∈ [l,r)
struct Node {
  int l, r;
  int x, y;
  int val, mx, mod;

  // value of empty set
  Node() : val(-INF), mx(-INF) {
    l = r = 0, mod = 0;
  }
  Node(int x, int val) : x(x), val(val), mx(val) {
    l = r = 0, mod = 0, y = rnd();
  }
};

Node nds[MAX];
int ndsz = 1; // nds[0] means empty

void push(int t) {
  if (!t || nds[t].mod == 0) return;
  nds[t].val += nds[t].mod;
  nds[t].mx += nds[t].mod;
  if (nds[t].l) nds[nds[t].l].mod += nds[t].mod;
  if (nds[t].r) nds[nds[t].r].mod += nds[t].mod;
  nds[t].mod = 0;
}

int getmx(int t) {
  push(t); // delete if sure (faster)
  return nds[t].mx;
}

void pull(int t) {
  if (!t) return;
  push(t), push(nds[t].l), push(nds[t].r); // must have
  nds[t].mx = max(nds[t].val, max(getmx(nds[t].l), getmx(nds[t
    ].r)));
}

pair<int, int> split(int t, int x) {
  if (!t) return {0, 0};
  push(t);
  if (x <= nds[t].x) {
    auto [l, r] = split(nds[t].l, x);
    nds[t].l = r;
    pull(t);
    return {l, t};
  } else {
    auto [l, r] = split(nds[t].r, x);
    nds[t].r = l;
    pull(t);
    return {t, r};
  }
}

int merge(int l, int r) {
  push(l), push(r);
  if (!l) return r;
  if (!r) return l;
  if (nds[l].y < nds[r].y) {
    nds[l].r = merge(nds[l].r, r);
    pull(l);
    return l;
  } else {
    nds[r].l = merge(l, nds[r].l);
    pull(r);
    return r;
  }
}

void insert(int &root, int x, int val) {
  nds[ndsz++] = Node(x, val);
  auto [l, r] = split(root, x);
  root = merge(merge(l, ndsz - 1), r);
}
```

```
// erase all equal to x
void erase(int &root, int x) {
  auto [lm, r] = split(root, x + 1);
  auto [l, m] = split(lm, x);
  root = merge(l, r);
}

// query [l, r)
int query(int &root, int ql, int qr) {
  auto [lm, r] = split(root, qr);
  auto [l, m] = split(lm, ql);
  int res = getmx(m);
  root = merge(merge(l, m), r);
  return res;
}

// update [l, r)
void update(int &root, int ql, int qr, int qx) {
  auto [lm, r] = split(root, qr);
  auto [l, m] = split(lm, ql);
  if (m) nds[m].mod += qx;
  root = merge(merge(l, m), r);
}
```

8.5 Персистентное ДД по неявному

```
struct Node;
int size(int);
int sum(int);

struct Node {
  int l, r;
  int val, sz, sm;

  Node() : l(0), r(0), val(0), sz(0), sm(0) {}
  Node(int val, int l, int r) : val(val), l(l), r(r) {
    sz = 1 + size(l) + size(r);
    sm = val + sum(l) + sum(r);
  }
};

Node nds[MAX];
int ndsz = 1;

int size(int t) { return nds[t].sz; }
int sum(int t) { return nds[t].sm; }

int newNode(int val, int l, int r) {
  nds[ndsz++] = Node(val, l, r);
  return ndsz - 1;
}

pair<int, int> split(int t, int x) {
  if (!t) return {0, 0};
  int szl = size(nds[t].l);
  if (szl >= x) {
    auto [l, r] = split(nds[t].l, x);
    int v = newNode(nds[t].val, r, nds[t].r);
    return {l, v};
  } else {
    auto [l, r] = split(nds[t].r, x - szl - 1);
    int v = newNode(nds[t].val, nds[t].l, l);
    return {v, r};
  }
}

bool chooseleft(int szl, int szr) {
  return rnd() % (szl + szr) < szl;
}

int merge(int l, int r) {
  if (!l) return r;
  if (!r) return l;
  if (chooseleft(nds[l].sz, nds[r].sz)) {
    int rr = merge(nds[l].r, r);
    int v = newNode(nds[l].val, nds[l].l, rr);
    return v;
  } else {
    int ll = merge(l, nds[r].l);
    int v = newNode(nds[r].val, ll, nds[r].r);
    return v;
  }
}

int insert(int root, int pos, int val) {
  int new_v = newNode(val, 0, 0);
  auto [l, r] = split(root, pos);
  return merge(merge(l, new_v), r);
}

int erase(int root, int pos) {
  auto [lm, r] = split(root, pos + 1);
  auto [l, m] = split(lm, pos);
  return merge(l, r);
}

// query [ql, qr), no new nodes
int get(int t, int ql, int qr) {
  if (!t || qr <= 0 || size(t) <= ql) return 0;
  if (ql <= 0 && size(t) <= qr) return sum(t);
  int lsz = size(nds[t].l);
  return get(nds[t].l, ql, qr)
       + (ql <= lsz && lsz < qr ? nds[t].val : 0)
       + get(nds[t].r, ql-lsz-1, qr-lsz-1);
}
```

8.6 Персистентное ДО

```
Node nds[MAX];
int ndsz = 1;
// nds[0] is default (empty) value

int sum(int v) { return nds[v].sm; }

// returns new root of subtree
int update(int v, int l, int r, int qi, int qx) {
  if (qi < l || r <= qi) return v;
  if (l + 1 == r) {
    nds[ndsz++] = Node(qx);
    return ndsz - 1;
  }
  int m = (l + r) / 2;
  int u = ndsz++;
  nds[u].l = update(nds[v].l, l, m, qi, qx);
  nds[u].r = update(nds[v].r, m, r, qi, qx);
  nds[u].sm = sum(nds[u].l) + sum(nds[u].r);
  return u;
}

int get(int v, int l, int r, int ql, int qr) {
  if (!v || qr <= l || r <= ql) return 0;
  if (ql <= l && r <= qr) return nds[v].sm;
  int m = (l + r) / 2;
  auto a = get(nds[v].l, l, m, ql, qr);
  auto b = get(nds[v].r, m, r, ql, qr);
  return a + b;
}
```

8.7 Спарсы

```
int tree[LOG][MAXN];
int floorlog2[MAXN+1]; // i ? (31 - __builtin_clz(i)) : 0

void build(vector<int> &a) {
  int n = a.size();
  copy(all(a), tree[0]);
  for (int i = 1; i < LOG; ++i) {
    int len = 1 << (i - 1);
    for (int j = 0; j + len < n; ++j)
      tree[i][j] = min(tree[i - 1][j], tree[i - 1][j + len]);
  }
  for (int i = 2; i <= n; ++i)
    floorlog2[i] = floorlog2[i / 2] + 1;
}

// min a[l..r)
int get(int l, int r) {
  int i = floorlog2[r - l];
  return min(tree[i][l], tree[i][r - (1 << i)]);
}
```

8.8 Фенвик

```
// Нумерация с 0
struct Fenwick {
  int n;
  vector<int> f;
  Fenwick(int n) : n(n), f(n+1) {}

  void build(const vector<int> &a) { // O(n)
    n=size(a), f.assign(n+1, 0);
    for (int i = 1; i <= n; ++i) {
      f[i] += a[i-1];
      int j = i + (i & -i);
      if (j <= n) f[j] += f[i];
    }
  }

  // a[i] += x
  void add(int i, int x) {
    for (++i; i <= n; i += i & -i) f[i] += x;
  }

  // sum a[0..i)
  int get(int i) {
    int ans = 0;
    for (; i > 0; i -= i & -i) ans += f[i];
    return ans;
  }

  // n > 0, a[..] >= 0,   pref[i] = a[0] + ... + a[i]
  // lower_bound по pref: min i, что pref[i] >= x (либо n)
  // get(lower_bound(x)) = максимальное, меньшее x (при x>0)
  int lower_bound(int x) {
    int i = 0;
```

```
    for (int b = 31 - __builtin_clz(n); b >= 0; --b) {
      int d = 1 << b;
      if (i+d <= n && f[i+d] < x) i+=d, x-=f[i];
    }
    return i;
  }
};

// sum a[x1..x2)[y1..y2)[z1..z2)
int sum_3d(int x1,int x2,int y1,int y2,int z1,int z2) {
  int ans = get(x2, y2, z2);
  ans -= get(x1,y2,z2) + get(x2,y1,z2) + get(x2,y2,z1);
  ans += get(x1,y1,z2) + get(x1,y2,z1) + get(x2,y1,z1);
  ans -= get(x1,y1,z1);
  return ans;
}

// a[l..r) += x
void update(int l, int r, int x) {
  T1.add(l, x);
  T1.add(r, -x);
  T2.add(l, -x * l);
  T2.add(r, x * r);
}
// sum a[0..i)
int get(int i) {
  return T1.get(i) * i + T2.get(i);
}
```